`KPop` : Unleash the full power of your *k*-mers!

`KPop` is a scalable method for the comparative analysis of microbial genomes and environmental samples; it transforms each input sequence or sample into a point (also called "vector" or "embedding") living in an abstract space with a moderate number of dimensions. `KPop` is based on full *k*-mer spectra and dataset-specific transformations; it allows to accurately compare hundreds of thousands of assembled, or thousands of unassembled microbial genomes or sequenced samples, in a matter of hours. It provides excellent resolution across a very large number of use cases and applications, even when the underlying genomic diversity is low. More details can be found in our Genome Biology paper.

Some easy-to-use Nextflow-orchestrated `KPop` workflows implemented by Ryan Morrison can be found here.

The `KPop` project gratefully acknowledges funding by several institutions, which allowed development to continue over a number of years:

The NIHR's HPRU GED
The European Union's NEARDATA
The Scottish Government's RESAS
The BBSRC for providing computing resources.

NIHR | Health Protection Research Unit in Genomics and Enabling Data at University of Warwick

NEARDATA

RESAS Rural & Environmental Science and Analytical Services

BBSRC

And finally, the awesome `KPop` logo was created by Emily Fotopoulou, to whom our heartfelt thanks go!

Table of contents

0. Installation

There are several possible ways of installing the software on your machine: through `conda`; by downloading pre-compiled binaries (Linux and MacOS x86_64 only); or manually.

> ⚠️ Note that the only operating systems we officially support are Linux and MacOS. ⚠️
>
> Both OCaml and R are highly portable and you might be able to manually compile/install everything successfully on other platforms (for instance, Windows). Please let us know if you succeed or if you encounter some unexpected behaviour. However, please note that in general we are unable to provide installation-related support or troubleshooting on specific hardware/software combinations.

0.1. Conda channel

BiOCamLib can be installed from the `bioconda` channel as package `kpop`. Just type

```
conda install -c bioconda kpop
```

or the equivalent command obtained replacing `conda` with `mamba` or `micromamba`, depending on which program you habitually use.

0.2. Pre-compiled binaries

You can download pre-compiled binaries for Linux and MacOS x86_64 from our releases. After doing so, just copy or move them to a directory which is accessible from your PATH.

For instance, supposing that you've downloaded programs to directory `~/.local/bin/`, in order to make them accessible from everywhere you'll have to execute a command such as

```
export PATH=~/.local/bin:$PATH
```

or add it to one of your login scripts (such as `~/.bashrc` or similar for `bash`).

Note that the binaries are generated according to the recipe described here.

0.3. Manual install

Alternatively, you can install `KPop` manually by cloning and compiling its sources. You'll need an up-to-date distribution of the OCaml compiler and the Dune package manager for that. Both can be installed through `opam`, the official OCaml distribution system. Once you have a working `opam` distribution you'll also have a working OCaml compiler, and Dune can be installed with the command

```
opam install dune
```

if it is not already present. Make sure that you install OCaml version 4.12 or later. Note that, although we don't expect major issues, we have never tested if the code compiles successfully with version 5 or above.

Cloning should be done with the option --recursive, as in

```
git clone --recursive git@github.com:PaoloRibeca/KPop.git
```

so as to make sure that all the subrepositories get correctly pulled.

Then go to the directory into which you have downloaded the latest `KPop` sources, and type

```
./BUILD
```

That should generate all the executables you'll need (as of this writing, `KPopCount`, `KPopCounterDB`, `KPopTwist_`, `KPopTwist`, `KPopTwistDB`) in the directory `./build`. Copy them to some favourite location in your PATH, for instance `~/.local/bin`.

For the time being, due to the presence of some legacy code, you'll also need to install some R packages (and possibly R itself if you don't already have it) by using your favourite R package manager. Those packages are:

```
data.table
ca
```

1. Quick start

Download the directory `Primer` from the directory `test` in the repository. Then enter it and run the following commands (throughout this documentation we'll assume that you're using a fairly up-to-date version of `bash`):

```
export K=5
date
KPopCount -k $K -f "" Train/Train.fasta -o Train-$K -v
KPopCountDB -i Train-$K -m Train/CLASS.txt -c CLASS -o /dev/stdout | KPopTwist -i /dev/stdin -o Classes-$K -v
KPopCount -k $K -f "" Test/Test.fasta -o /dev/stdout | KPopTwistDB -i T Classes-$K -t /dev/stdin -d Classes-$K Test-$K -v
echo -n ">>> Misclassified sequences: "; cat Test-$K.KPopSummary.txt | awk -F '\t' 'BEGIN{count=0} {split($1,s,"-"); if (s[2]!=$6) ++count}
        END{print count}'
date
```

That should produce an output such as this one:

```
Thu 11 Dec 20:49:53 GMT 2025
╔═══════════════════════════════════════════╗
┃                                           ┃
┃  This is KPopCount version 28 [08-Dec-2025]  ┃
┃                                           ┃
╚═══════════════════════════════════════════╝
 | compiled against: BiOCamLib version 481 [10-Dec-2025];
                     KPop version 740 [11-Dec-2025]
 | (c) 2017-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
(BiOCamLib__KMers.Iterator.Hasher.make): Initializing small encoder (bits=2, k=5)
(Dune__exe__KPopCount.(fun)): Hashed and added 500 spectra from 1000 reads and 1 file.
(KPop__KMerDB_Base.to_binary): Outputting database to file 'Train-5.KPopSpectra'... done.
╔═══════════════════════════════════════════╗
┃                                           ┃
┃  This is KPopTwist version 30 [24-Oct-2025]  ┃
┃                                           ┃
╚═══════════════════════════════════════════╝
 | compiled against: BiOCamLib version 481 [10-Dec-2025];
                     KPop version 740 [11-Dec-2025]
 | (c) 2022-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
Thu 11 Dec 2025 20:49:53 GMT: [1/16] Exporting table...
Thu 11 Dec 2025 20:49:54 GMT: [2/16] Reading table...
Thu 11 Dec 2025 20:49:54 GMT: [3/16] Splitting table...
Thu 11 Dec 2025 20:49:54 GMT: [4/16] Discarding k-mers...
Thu 11 Dec 2025 20:49:54 GMT: [5/16] Resampling k-mers...
Thu 11 Dec 2025 20:49:54 GMT: [6/16] Thresholding k-mers...
Thu 11 Dec 2025 20:49:54 GMT: [7/16] Normalizing counts...
Thu 11 Dec 2025 20:49:54 GMT: [8/16] Twisting counts...
Thu 11 Dec 2025 20:49:54 GMT: [9/16] Writing twisted...
Thu 11 Dec 2025 20:49:54 GMT: [10/16] Writing inertia...
Thu 11 Dec 2025 20:49:54 GMT: [11/16] Normalizing twister...
Thu 11 Dec 2025 20:49:54 GMT: [12/16] Transposing twister...
Thu 11 Dec 2025 20:49:54 GMT: [13/16] Writing twister...
Thu 11 Dec 2025 20:49:54 GMT: [14/16] Encoding twisted...
Thu 11 Dec 2025 20:49:54 GMT: [15/16] Encoding twister...
Thu 11 Dec 2025 20:49:55 GMT: [16/16] Cleaning up...
Thu 11 Dec 2025 20:49:55 GMT: All done.
╔═══════════════════════════════════════════╗
┃                                           ┃
┃  This is KPopTwistDB version 47 [09-Nov-2025]  ┃
┃                                           ┃
╚═══════════════════════════════════════════╝
 | compiled against: BiOCamLib version 481 [10-Dec-2025];
                     KPop version 740 [11-Dec-2025]
 | (c) 2022-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
       2024      Ünsal Öztürk <uensal.oeztuerk@gmail.com>
(KPop__Twister.of_binary): Reading twister from file 'Classes-5.KPopTwister'... done.
(KPop__KMerDB_Base.of_binary): Reading database from file '/dev/stdin'... done.
(KPop__Twister.add_twisted_from_database): Twisting spectra: done 500/500.
(KPop__Twisted.of_binary): Reading vectors from file 'Classes-5.KPopTwisted'... done.
(KPop__Twisted.summarize_distances_rowwise): Writing distance digest to file '/dev/stdout': done 500/500 rows.
>>> Misclassified sequences: 0
Thu 11 Dec 20:49:55 GMT 2025
```

This is an example of `KPop`-based classifier. The original input FASTA file, which is the result of a phylogenetic simulation, contains 1000 sequences having names such as `S2-C1`, meaning "sequence 2 belonging to class 1". There are 10 different classes and 100 sequences per class. In the following sections we will see in more detail how `KPop` can be used to classify sequences, but here, to give a quick summary:

1. Sequences with an odd index are taken to be part of the training set, sequences with an even index are considered part of the test set
2. An additional file is provided, specifying the class labels for each sequence belonging to the training set as the content of a metadata field named `CLASS`. This information, the class labels, is the only input that `KPop` needs to generate a classifier in addition to the sequences

3. The command

```
export K=5; KPopCount -k $K -f "" Train/Train.fasta -o Train-$K -v
```

runs `KPopCount` (with *k*=5) on the training sequences and computes a separate "spectrum" (i.e., a vector of *k*-mer occurrencies) for each sequence; the result is stored in a table called `Train-5.KPopSpectra`. Although the table is stored in binary format, you could print it out as text and look into its content (both counts and metadata) with the command

```
KPopCountDB -i Train-$K -O /dev/stdout | less
```

Note that while the complete name of the *k*-mer database is `Train-5.KPopSpectra`, you just refer to it as to `Train-5` — the `.KPopSpectra` extension is autmatically added by the programs and does not need to be specified explicitly

4. In the command

```
KPopCountDB -i Train-$K -m Train/CLASS.txt -c CLASS -o /dev/stdout | KPopTwist -i /dev/stdin -o Classes-$K -v
```

`KPopCountDB` loads the database we just generated, adds to it as metadata the sequence labels present in file `CLASS.txt`, and combines all the sequences belonging to each class into a single representative. It then outputs the results to standard output and passes them through a pipe to program `KPopTwist`, which "twists" the class representatives into vectors (i.e., points) belonging to a reduced-dimensionality space. Both the twisted spectra and the "twister" — i.e., the operator that can be used to convert the original spectra to twisted space — are stored as binary tables (files `Classes-5.KPopTwisted` and `Classes-5.KPopTwister`, respectively) that can be reused later on. As before, we did not need to explicitly specify file extensions; and as before, we could look into the binary files and convert them into plain text tables. For instance, we could visualise the twisted vectors by saying

```
KPopTwistDB -i t Classes-$K -O t /dev/stdout | less
```

5. The command

```
KPopCount -k $K -f "" Test/Test.fasta -o /dev/stdout | KPopTwistDB -i T Classes-$K -t /dev/stdin -d Classes-$K Test-$K -v
```

uses `KPopCount` to produce a separate spectrum for each test sequence in `Test.fasta`, sending them through a pipe to `KPopTwistDB`; `KPopTwistDB` then twists them according to the twister generated at the previous stage (named `Classes-5.KPopTwisted` and referred to on the command line simply as to `Classes-5`) and computes distances between the twisted test sequences and the class representatives stored in `Classes-5.KPopTwisted` (which, again, we refer to only as to `Classes-5` without having to explicitly specify an extension). The results are output to a summary plain-text file, which gets automatically named `Test-5.KPopSummary.txt`. It contains information about the two closest classes for each sequence

6. Finally, the command

```
echo -n ">>> Misclassified sequences: "; cat Test-$K.KPopSummary.txt | awk -F '\t' 'BEGIN{count=0} {split($1,s,"-"); if (s[2]!=$6)
        ++count} END{print count}'
```

parses the classification results present in `Test-5.KPopSummary.txt` and computes the number of misclassified sequences, of which there appears to be none.

Congratulations! You have now moved your first steps in the fascinating world of `KPop`.

1.2. Visualisation of Quick Start results

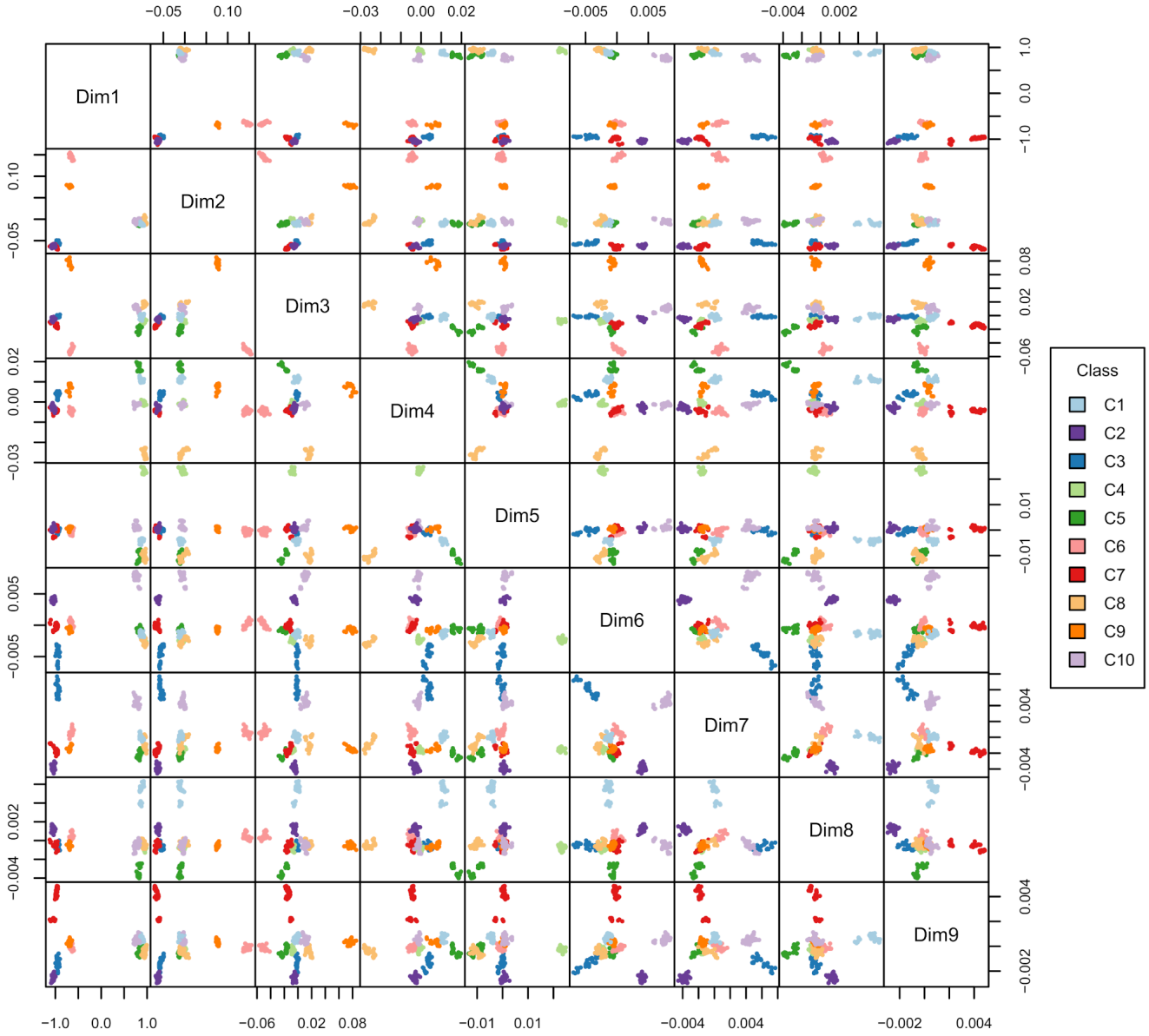
Now we can visualise the embeddings (a.k.a. twisted spectra, or multidimensional vectors in `KPop` space) generated by `KPop` for each test sequence as a scatterplot matrix for all possible 2-dimensional projections of the `KPop` space. This normally wouldn't be recommended as there will likely be a large number of dimensions, but in this example we only have 9.

To do this, we firstly create a `Test.5.KPopTwisted.txt` file using KPopTwistDB with only the test sequences:

```
cat clusters-small.fasta | awk -v K="$K" '{nr=(NR-1)%4; if (nr==2) split($0,s,"[>-]"); if (nr==3) print ">"s[2]"-"s[3]"\n"$0}' | KPopCount
        -k $K -L -f /dev/stdin | KPopTwistDB -i T Classes.$K -k /dev/stdin -o t /dev/stdout -v | KPopTwistDB -i t /dev/stdin -O t Test.$K
```

We then visualise the results as a scatterplot matrix using `R`:

▶ Script to visualise Quick Start results



One can see how `KPop` is able to embed sequences (i.e., to map them to an abstract multi-dimensional space) in such a way that they cluster according to their original colouring — which represents, in this case, the ground truth. As the clusters are well-defined and have neat boundaries along a suitable number of dimensions, as shown by the projections, that results in a good (actually perfect) classifier for this example if the cluster/class centroids are considered as reference points and the closest centroid is used to predict the class each sequence belongs to. However, as `KPop` is able to explicitly turn sequences into vectors, any kind of more complex classifier based on machine learning or AI techniques (for instance *k*-NN, decision trees, random forests or others) can also be used. We provide some examples later on.

As mentioned previously, this plot is a great way to visually check that test sequences cluster in the correct class — however, it wouldn't be as useful for a dataset with 1000 dimensions!

2. Frequently asked questions

There are no binaries available for my OS, what do I do?

`KPop` is almost entirely implemented in OCaml, an industry-strength functional statically-typed programming language that offers remarkable concision, symbolic power, robustness, compiled speed, and portability. Due mostly to historical reasons, a small part of `KPop` is still written in R, although we hope to eventually migrate everything to OCaml.

Both OCaml and R are highly portable and you might be able to manually compile/install everything successfully on other platforms. See the Installation section below.

How does `KPop` compare with minimiser-based methods?

`KPop` is much more sensitive than minimiser-based methods, allowing out-of-the-box accurate comparison of assembled or unassembled long sequences irrespective of whether they differ by single nucleotides or by large portions. This is usually not true for minimiser-based methods no matter which hashing scheme they use, as they only produce a much coarser comparison. A number of benchmarks and a detailed explanation of the differences between the two approaches can be found in our Genome Biology paper.

Crucially, while minimiser-based methods are geared towards providing distances between sequences, `KPop` explicitly produces *embeddings*, i.e., it is able to turn sequences into points of a low-dimensionality latent space. Having an explicit latent space is important for many reasons — it helps explainability; it also makes it possible to perform

direct clustering and use vector DBs or libraries to store and search embedded sequences.

Can I run it on my laptop?

Depending on the problem at hand, `KPop` analysis can require a relatively large amount of resources, memory in particular. That is a conscious design choice — one could not leverage the power of full *k*-mer spectra otherwise. If you need to explore datasets with millions of sequences, we recommend that you do so on a robustly sized HPC node. However, not everything is expensive — for instance, and similar to what happens with other big-data frameworks, while the "training" phase needed to create `KPop`-based classifiers is typically resource-intensive, the classifiers themselves can typically be run on lower-end machines.

All `KPop` programs are parallelised and will automatically use as many CPUs as are available on your machine, in order to reduce as much as possible the wallclock time taken by your tasks.

How do I choose *k*?

As explained in our Genome Biology paper, the choice of *k* can be tricky, and you should always make sure you try different results and select the lowest value of *k* for which results become stable. One empirical criterion that might help you in most cases is to choose

$$k := \mathrm{argmax}_k \mathrm{min}_s \frac{\mathrm{max}_h C_{hs}^k}{\left(\prod_h C_{hs}^k \right)^{1/n_h}}$$

where C_{hs}^k is a non-zero count for *k*-mer h in sample s and there are n_h non-zero *k*-mers. I.e., you would select the k maximising the minimum across samples of the ratio between the maximum non-zero count and the (harmonic) mean of the non-zero counts for each sample.

This is too complicated. Help please!

We do realise that a method such as `KPop` might look a bit less straightforward than others that are popular in the domain of bioinformatics. On the other hand, `KPop` is also very versatile and can be applied to a number of radically different use cases, which usually pays off once you have managed to go past the initial learning curve. As for us, we constantly strive to simplify usage and command lines with every new revision and version, so that all programs become progressively less cluttered with options and easier to use. We believe a good demonstration of this strategy can be seen in the upcoming version 2 programs — despite them containing significantly more functionality than version 1, syntax has been drastically simplified and the main `KPop` use cases can now be implemented with a few simple commands.

If you still think that all this is too complicated for you, do not despair! Some easy-to-use Nextflow-orchestrated `KPop` workflows can be found here; they implement typical use cases end-to-end and might be sufficient for what you need to do. We will add more in the future, as the need for them becomes clearer and their implementation stabilises.

3. Overview of commands

`KPop` is implemented as a suite of different programs, which can be combined into complex workflows in a modular fashion. They are:

- `KPopCount`. It implements extraction of *k*-mer spectra from files containing sequences, sequencing reads or general text (in FASTA format; in both single- and paired-end FASTQ format; in tabular format). Several encoding modes (alphabet- and dictionary-based) and *k*-mer extraction schemes (plain and gapped) are supported as of version 2. The resulting collections of *k*-mer spectra are exported as binary databases
- `KPopCountDB`. It implements complex manipulations on binary databases of *k*-mer spectra, so that the user can: operate tranformations on counts; annotate counts with metadata; combine sets of spectra; export databases to text tables; and more
- `KPopTwist`. It implements the unsupervised generation of coordinate transformations (or "twisters") from databases of *k*-mer spectra. Each transformation is optimised for the database at hand, and turns ("twists") *k*-mer spectra into numerical vectors of a typically very much reduced dimensionality. Both the transformation and the resulting twisted spectra can be stored as a binary object for future use
- `KPopTwistDB`. It implements a number of operations on twisted spectra. It can: use an existing twister to twist *k*-mer spectra; generate databases of twisted spectra and output/input them as binary files or text tables; compute and summarise distances between twisted spectra; and more.

3.1. General design

While `KPopTwist` implements a one-shot operation (generating a twister), `KPopCount`, `KPopCountDB` and `KPopTwistDB` need to be versatile and perform a number of complex tasks, ranging from householding management of spectra and twisted spectra to complex operations that are an essential component of larger workflows.

In response to such needs, all programs apart from `KPopTwist` are able to execute a series of *actions*. Actions are specified each one as a different command line option, and performed in order of specification — i.e., saying something like `KPopCountDB -D -N` is *not* the same as saying `KPopCountDB -N -D`.

In addition, `KPopCount`, `KPopCountDB` and `KPopTwistDB` have *registers*, i.e., slots to/from which the contents of external files and the results of computations can be loaded/saved. You can see registers as program variables, each one having a specific type. For instance, the `KPop` programs know that only twisters can be loaded to/saved from twister registers, and will enforce that by requiring and checking that your external file has the correct extension (`.KPopTwister`) and format when doing I/O to/from a twister register.

`KPopCount` has one register:

- a *database* register, holding a set of *k*-mer spectra (i.e., a set of *k*-mers and their associated frequencies). There is one spectrum per sample, and more spectra can be added to the database as input files are processed.

`KPopCountDB` has two registers:

- a *database* register, holding a set of *k*-mer spectra (i.e., a set of *k*-mers and their associated frequencies). There is one spectrum per sample, and additional metadata in the form of strings or numbers can be associated to the samples (see our Genome Biology paper for more information). Due to space optimisation reasons, modifications of the database happen in-place
- a *selection* register, holding a set of sample labels. Labels can be specified on the command line or read from the database present in the database register. Collective operations (selection, deletion, combination, and so on) can be performed on the samples present in the database register according to the set of sample names specified by the selection register, which acts as a guide to enable complex operations on the database. For instance, one might extract from the database all the sample names matching some regular expression and put them in the selection register, add some more sample names to the selection register, and finally delete from the database all the samples whose names do not appear in the selection register. (Note that collective operations on the spectra can also be specified through metadata fields.)

`KPopTwistDB` has two registers:

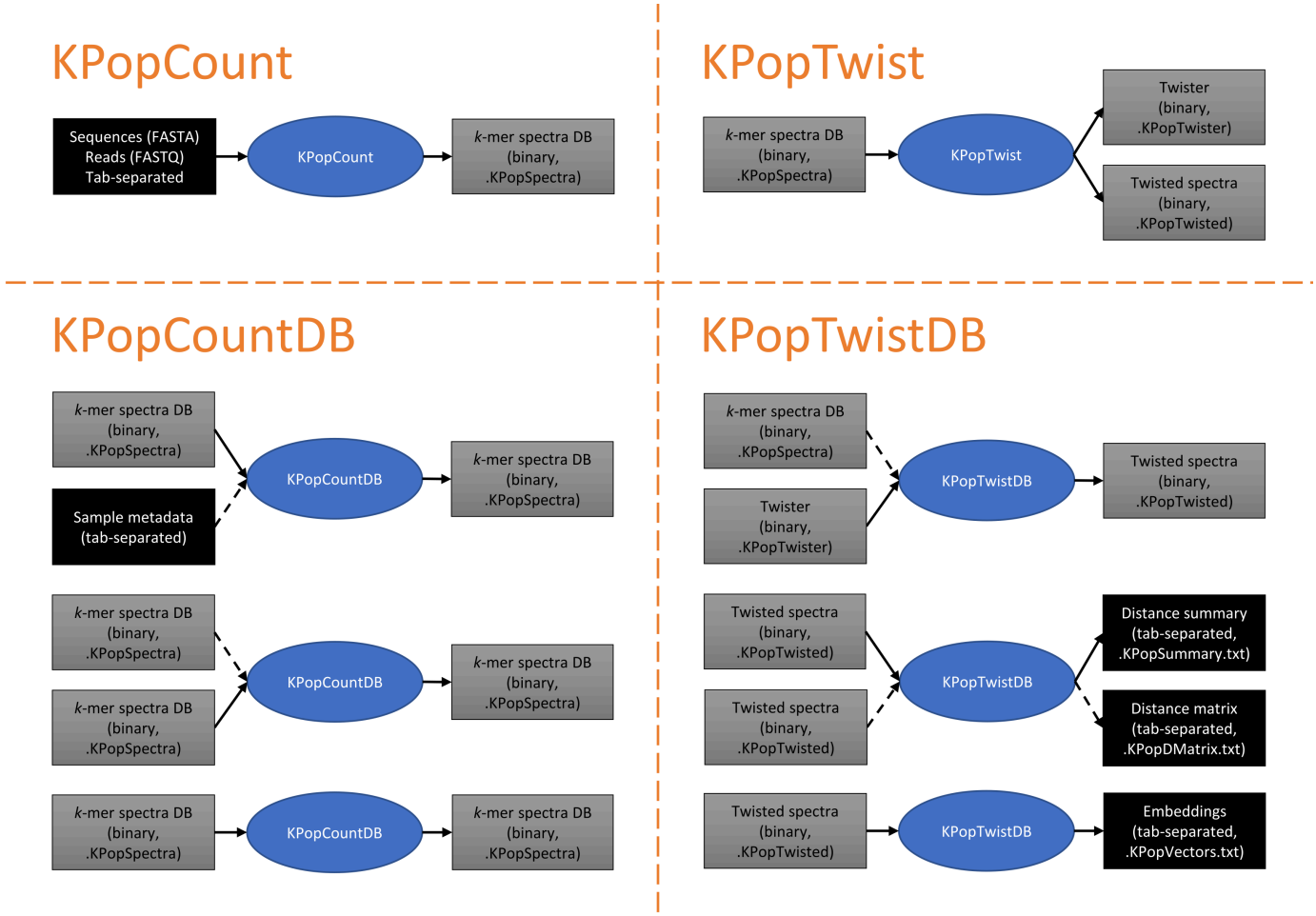
- a *twister* register holding a twister, i.e., an object able to transform *k*-mer spectra to "twisted" embeddings/vectors in an abstract `KPop` space (note that the transformation is dataset-dependent and generated by the user when runnning `KPopTwist` on a set of spectra, as explained in our Genome Biology paper and the next section)
- a *twisted* register holding "twisted" vectors, i.e., `KPop` embeddings of biological sequences into an abstract `KPop` space.

Options are provided to write content from/to these registers, both in binary and textual form — the latter can be used, for instance, to import/export data from/to `R`. In addition to input/output to/from registers, both `KPopCountDB` and `KPopTwistDB` are able to read and process external database files containing *k*-mer spectra in the format produced by `KPopCount` — this ensures that new spectra/twisted spectra can be added to their respective registers. A figure showing some of the available data transformations and formats can be found in the next section.

In general, registers provide "memory" (or a "state" in computer science parlance) to `KPopCount`, `KPopCountDB` and `KPopTwistDB` — by using them and the rich set of command line options we implemented, one can easily combine `KPop` programs and build complex workflows performing sophisticated tasks (see the section on examples below).

3.2. Building workflows

The following figure summarises the main typical data transformations that are possible and the file formats used to encode inputs and outputs in each case:



There are many more possible ways of using `KPopCountDB` and `KPopTwistDB`. For instance, with `KPopCountDB` one can compute distances between untwisted *k*-mer spectra; with `KPopTwistDB` one can accumulate twisted *k*-mer spectra or distances into existing databases, and convert binary files from/to plain-text tab-separated tables. In the latter case, as explained, automatic naming rules are applied depending on the type of the register being used — for example, a command such as

```
KPopTwistDB ... -o t Classes
```

would result in the production of a binary file named `Classes.KPopTwisted` containing twisted *k*-mer spectra, while the command

```
KPopTwistDB ... -O t Classes
```

will produce the same results but output them to a tab-separated tabular file named `Classes.KPopTwisted.txt`. In general, in the style of BLAST indices, you do not have to worry about formats or extensions as long as you consistently use the same prefix (in this case, `Classes`) to indicate logically related files.

Note that as a rule, sequence/sample names, as well as metadata field names and field values, cannot contain double quote `'"'` or tabulation `'\t'` characters.

4. Command line syntax

4.1. `KPopCount`

This is the list of command line options available for the program `KPopCount`. You can visualise the list by typing

```
KPopCount -h
```

in your terminal. You will see a header containing information about the version:

```
This is KPopCount version 29 [10-Jan-2026]
 compiled against: BiOCamLib version 498 [10-Jan-2026];
                   KPop version 767 [10-Jan-2026]
 (c) 2017-2026 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form the command can be used is:

```
KPopCount [ACTIONS]
```

Actions. They are executed delayed and in order of specification.

Input/Output of spectra databases:

Option	Argument(s)	Effect	Note(s)
`-0` `--zero` `--empty`		load an empty database into the register	
`-i` `--input`	*binary_file_prefix*	load into the register the database present in the specified file (which must have extension `.KPopSpectra` unless file is `/dev/*`)	
`-o` `--output`	*binary_file_prefix*	save the database present in the register to the specified file (which will be given extension `.KPopSpectra` unless file is `/dev/*`)	

Algorithmic parameters:

Option	Argument(s)	Effect	Note(s)
`-k` `--k-mer-size` `--k-mer-length`	*positive_integer*	set the hashing strategy to iteration over regular *k*-mers and specify the *k*-mer length to be used. Options `-k` and `-g` are mutually exclusive; if multiple are specified, the last one will take effect	default= *continuous k-mers of size 12*
`-g` `--gapped-k-mer-sizes` `--gapped-k-mer-lengths`	*BLOCK_SIZE GAP_SIZE*	where *BLOCK-SIZE := positive_integer* *GAP-SIZE := positive_integer* Set the hashing strategy to iteration over symmetrical gapped *k*-mers (having a *BLOCK-GAP-BLOCK* structure, with *BLOCK*-s of the same size) and specify their geometry in terms of *BLOCK* and *GAP* sizes, respectively. For instance, option `-g 5 1` will iterate on all existing *k*-mers of size 11 (5+1+5) and not take the central nucleotide into account for the purpose of computing the hash. Options `-k` and `-g` are mutually exclusive; if multiple are specified, the last one will take effect	default= *not used*
`-c` `--content`	`ss-DNA` | `single-stranded-DNA` | `ds-DNA` | `double-stranded-DNA` | `protein` | *FULL*	set how contents of following input files should be interpreted. When content is `ss-DNA`, `protein` or `text`, only the sequence is hashed; when content is `ds-DNA`, both sequence and reverse complement are hashed. `ss-DNA` prevents automatic matching of reverse-complemented sequences; use it only when comparing a set of single, homogeneus sequences. These are shortcuts for the full form of this option, which is defined as *FULL :=* | `DNA(` *STRANDEDNESS* `,` *CASE_SENSITIVITY* `,` *UNKNOWN_CHAR_ACTION* `)` | `protein(` *UNKNOWN_CHAR_ACTION* `)` | `text(` *CASE_SENSITIVITY* `,` *UNKNOWN_CHAR_ACTION* `,` *dictionary_file_name* `)` where *STRANDEDNESS :=* `ss` | `single-stranded` | `ds` | `double-stranded` *CASE_SENSITIVITY :=* `ci` | `case-insensitive` | `cs` | `case-sensitive` *UNKNOWN_CHAR_ACTION :=* `split` | `ignore` | `error` If `case-insensitive` is specified, DNA/protein sequences are converted to uppercase characters, while text sequences (and dictionary entries) are converted to lowercase characters. *UNKNOWN_CHAR_ACTION* decides what happens when an unknown character is found in the input. Option `split` (the default for DNA/protein sequences) splits the input sequence and skips unknown characters (for instance `N` / `X`) whenever they are encountered; option `ignore` (the default for text) silently skips unknown characters (for instance whitespace); option `error` causes the program to abort. If a dictionary file is specified, each of its lines is interpreted as a different dictionary entry/token	default= `DNA(double-stranded,case-insensitive,split)`
`-w` `--weights` `--weights-from-sequence-names`	*non_negative_integer*	given the index *n* specified as a parameter, extract the *n*-th number from each sequence name and weigh the corresponding sequence accordingly. Indices are 1-based; a value of `0` disables weighting. If no such field exists, the program will fail. If the weight is a float number, the ceiling of such number will be used	default= *do not weigh*

Input/Output of sequences for processing:

Option	Argument(s)	Effect	Note(s)
`-f` `--fasta`	*label fasta_file_name*	process the sequences contained in the specified FASTA input file. If a label is specified, the hashes extracted from all the sequences are collected into one spectrum having the label as name; if the label is empty, each sequence is turned into one separate spectrum and the sequence name is used as label. Label and sequence names must not contain double quote `"` characters. While you can specify several inputs possibly having different formats, contents are expected to be homogeneous across inputs	
`-s` `--single-end`	*label fastq_file_name*	process the sequences contained in the specified FASTQ input file containing single-end sequencing reads. If a label is specified, the hashes extracted from all the sequences are collected into one spectrum having the label as name; if the label is empty, each sequence is turned into one separate spectrum and the sequence name is used as label. Label and sequence names must not contain double quote `"` characters. While you can specify several inputs possibly having different formats, contents are expected to be homogeneous across inputs	
`-p` `--paired-end`	*label fastq_file_name1 fastq_file_name2*	process the sequences contained in the specified FASTQ input file containing paired-end sequencing reads. If a label is specified, the hashes extracted from all the sequences are collected into one spectrum having the label as name; if the label is empty, each sequence is turned into one separate spectrum and the sequence name is used as label. Label and sequence names must not contain double quote `"` characters. While you can specify several inputs possibly having different formats, contents are expected to be homogeneous across inputs	
`-t` `--tabular`	*label tabular_file_name*	process the sequences contained in the specified tabular input file. If a label is specified, the hashes extracted from all the sequences are collected into one spectrum having the label as name; if the label is empty, each sequence is turned into one separate spectrum and the sequence name is used as label. Label and sequence names must not contain double quote `"` characters. While you can specify several inputs possibly having different formats, contents are expected to be homogeneous across inputs	

Miscellaneous options. They are set immediately

Option	Argument(s)	Effect	Note(s)
`-T` `--threads`	*computing_threads*	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *4*
`-v` `--verbose`		set verbose execution	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

4.2. `KPopCountDB`

This is the list of command line options available for the program `KPopCountDB`. You can visualise the list by typing

```
KPopCountDB -h
```

in your terminal. You will see a header containing information about the version:

```
This is KPopCountDB version 55 [10-Dec-2025]
 compiled against: BiOCamLib version 498 [10-Jan-2026];
                   KPop version 767 [10-Jan-2026]
 (c) 2020-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form the command can be used is:

```
KPopCountDB [ACTIONS]
```

Actions. They are executed delayed and in order of specification.

Actions on the database register — Input/Output operations:

Option	Argument(s)	Effect	Note(s)
`-0` `--zero` `--empty`		load an empty database into the register	
`-i` `--input`	*binary_file_prefix*	load into the register the database present in the specified binary file (which must have extension `.KPopSpectra` unless file is `/dev/*`)	
`-I` `--Input`	*tabular_file_prefix*	load into the register the database present in the specified tabular files (which must have extensions `.KPopKMatrix.txt` and `.KPopMMatrix.txt` unless file is `/dev/*`)	
`--addition-criterion` `--database-addition-criterion`	`first` | `second` | `union` | `intersect`	set the criterion used to combine databases of spectra. Possibilities are: `first` The resulting database will have the same *k*-mer and metadata labels as the first database `second`	default= `union`

Option	Argument(s)	Effect	Note(s)
		The resulting database will have the same *k*-mer and metadata labels as the second database `union` The resulting database will have as *k*-mer and metadata labels the union of the *k*-mer and metadata labels of the two databases `intersect` The resulting database will have as *k*-mer and metadata labels the intersection of the *k*-mer and metadata labels of the two databases. For criteria `first`, `second`, and `union`, missing data will be set to zero in the case of *k*-mer counts, and to the empty string in the case of metadata entries	
`-a` `--add` `--add-database`	*binary_file_prefix*	add to the register the database present in the specified binary file (which must have extension `.KPopSpectra` unless file is `/dev/*`)	
`-m` `--metadata` `--add-metadata`	*metadata_table_file_name*	add to the register metadata from the specified tabular file. Metadata should be presented as a tab-separated text table, with a header containing spectrum labels and with row names being metadata fields labels. Spectrum labels, metadata field names and metadata values must not contain double quote `"` characters	
`--summary`		print a summary of the database present in the register	
`-o` `--output`	*binary_file_prefix*	save the database present in the register to the specified file (which will be given extension `.KPopSpectra` unless file is `/dev/*`)	
`--precision`	*positive_integer*	set the number of precision digits to be used for tabular output	default= *15*
`--Output-zero-kmers` `--Output-zero-k-mers`	`true` | `false`	whether to output *k*-mers whose frequencies are all zero when writing the database as tabular files	default= `true`
`-O` `--Output`	*tabular_file_prefix*	write the database present in the register as tab-separated files. File extensions are automatically assigned (will be `.KPopKMatrix.txt` and `.KPopMMatrix.txt`, unless file is `/dev/*`)	

Actions on the database register — Other operations:

Option	Argument(s)	Effect	Note(s)
`--combination-criterion` `--spectrum-combination-criterion`	`mean` | `median`	set the criterion used to combine the *k*-mer frequencies of selected spectra. To avoid rounding issues, each *k*-mer frequency is also rescaled by the largest normalization across spectra (`mean` averages frequencies across spectra; `median` computes the median across spectra)	default= *mean*
`-c` `--combine` `--combine-by-class` `--combine-spectra-by-class`	*<classes_metadata_field_name>*	split the database into classes according to the labels contained in the specified metadata field and combine the spectra belonging to each class into a separate vector named as the class label. Delete original spectra. Class label cannot be the same as the name of an existing spectrum	
`--transform`	*TRANSFORMATION*	replace the database with the one obtained from the specified transformation. Transformations are defined as follows: *TRANSFORMATION* := | `threshold(` *non-negative_float* `)` | `power(` *float* `)` | `binary` | `clr` | `pseudocounts(` *POWER* `,` *QUANTIZE* `)` *POWER* := *non-negative_float* *QUANTIZE* := `false` | `true` A value such that 0. ≤ *THRESHOLD* < 1. is interpreted as a fraction relative to the sum of all the counts in the spectrum; values such that *THRESHOLD* ≥ 1. are considered absolute thresholds; `binary` is an alias for `power(0)`. For the exact definition of transformations `clr` and `pseudocounts`, see https://doi.org/10.1186/s13059-025-03585-8	default= `power(1)`
`-d` `--distill` `--distill-kmers`	*classes_metadata_field_name summary_file_prefix*	optimize *k*-mers by identifying which ones are most informative according to the labels contained in the specified metadata field and by re-sorting *k*-mers in decreasing order accordingly. The labels must identify at least two equivalence classes, and fewer classes than the number of *k*-mers. Details of the procedure will be written to the specified summary file (which will be given extension `.KPopDistill.txt` unless file is `/dev/*`)	
`--distance` `--distance-function`	`euclidean` | `minkowski(` *non-negative_float* `)`	set the function to be used when computing distances. The parameter for `minkowski()` is the power	default= `euclidean`
`--distance-normalize`	`true` | `false`	whether spectra should be normalized prior to computing distances	default= *true*

Option	Argument(s)	Effect	Note(s)
`--distance-normalization`			
`--distances` `--compute-distances` `--compute-spectral-distances`	*REGEXP_SELECTOR* *REGEXP_SELECTOR* *binary_file_prefix*	where *REGEXP_SELECTOR := metadata_field ~ regexp[, ... , metadata_field ~ regexp]* and regexps are defined according to https://ocaml.org/api/Str.html: select two sets of spectra from the register and compute and output distances between all possible pairs (metadata fields must match the regexps specified in the selector; an empty metadata field makes the regexp match labels. The result will be given extension `.KPopDMatrix` unless file is `/dev/*`)	

Actions involving the selection register:

Option	Argument(s)	Effect	Note(s)
`-L` `--labels` `--selection-from-labels`	*spectrum_label[, ... , spectrum_label]*	put into the selection register the specified labels	
`-R` `--regexps` `--selection-from-regexps`	*metadata_field ~ regexp[, ... , metadata_field ~ regexp]*	put into the selection register the labels of the spectra whose metadata fields match the specified regexps and where regexps are defined according to https://ocaml.org/api/Str.html. An empty metadata field makes the regexp match labels	
`-A` `--add-combined-selection` `--selection-combine-and-add`	*spectrum_label*	combine the spectra whose labels are in the selection register and add the result (or replace it if a spectrum named *spectrum_label* already exists) to the database present in the database register	
`-D` `--delete` `--selection-delete`		drop the spectra whose labels are in the selection register from the database present in the register	
`-N` `--selection-negate`		negate the labels that are present in the selection register	
`-P` `--selection-print`		print the labels that are present in the selection register	
`-C` `--selection-clear`		purge the selection register	

Miscellaneous options. They are set immediately

Option	Argument(s)	Effect	Note(s)
`-T` `--threads`	*computing_threads*	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *nproc*
`-v` `--verbose`		set verbose execution	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

4.3. `KPopTwist`

This is the list of command line options available for the program `KPopTwist`. You can visualise the list by typing

```
KPopTwist -h
```

in your terminal. You will see a header containing information about the version:

```
This is KPopTwist version 30 [24-Oct-2025]
 compiled against: BiOCamLib version 481 [10-Dec-2025];
                   KPop version 740 [11-Dec-2025]
 (c) 2022-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
```

followed by detailed information. The general form the command can be used is:

```
KPopTwist -i|--input <binary_input_prefix> -o|--output <binary_output_prefix> [OPTIONS]
```

Algorithmic parameters

Option	Argument(s)	Effect	Note(s)
`--keep` `--keep-kmers` `--kmers-keep`	*kmer_list_file*	discard the *k*-mers not listed in this file before twisting the table. The file must contain one *k*-mer label per line and no header	default= *keep all*
`--sample` `--sample-kmers` `--kmers-sample`	*fractional_float*	fraction of the *k*-mers to be randomly resampled and kept after parameter `-k` has been applied and before twisting	default= *1.*
`--kmers-threshold`	*non-negative_integer*	compute the sum of all transformed (and possibly normalized) counts for each *k*-mer, and eliminate *k*-mers such that the corresponding sum is less than the largest sum rescaled by this threshold. This filters out *k*-mers having low frequencies across all spectra	default= *0.*

Input/Output

Option	Argument(s)	Effect	Note(s)
`-i` `--input`	*binary_file_prefix*	load the specified *k*-mer database in the register and twist it. File extension is automatically determined (will be `.KPopSpectra` unless file is `/dev/*`)	*(mandatory)*
`-o` `--output`	*binary_file_prefix*	use this prefix when saving generated twister and twisted sequences. File extensions are automatically determined (will be `.KPopTwister` and `.KPopTwisted` unless file is `/dev/*`)	*(mandatory)*
`-k` `--output-kmers` `--output-twisted-kmers`	*binary_file_prefix*	use this prefix when saving generated twisted *k*-mers. File extension is automatically determined (will be `.KPopTwisted` unless file is `/dev/*`)	default= *do not output*

Miscellaneous

Option	Argument(s)	Effect	Note(s)
`-T` `--threads`	*computing_threads*	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *nproc*
`--keep-temporaries`		keep temporary files rather than deleting them in the end	default= *do_not_keep*
`-v` `--verbose`		set verbose execution	default= *quiet_execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

4.4. `KPopTwistDB`

This is the list of command line options available for the program `KPopTwistDB`. You can visualise the list by typing

```
KPopTwistDB -h
```

in your terminal. You will see a header containing information about the version:

```
This is KPopTwistDB version 47 [09-Nov-2025]
 compiled against: BiOCamLib version 481 [10-Dec-2025];
                   KPop version 740 [11-Dec-2025]
 (c) 2022-2025 Paolo Ribeca <paolo.ribeca@gmail.com>
     2024      Ünsal Öztürk <uensal.oeztuerk@gmail.com>
```

followed by detailed information. The general form the command can be used is:

```
KPopTwistDB [ACTIONS]
```

Actions. They are executed delayed and in order of specification.

Actions on the database registers — Input/Output operations:

Option	Argument(s)	Effect	Note(s)
`-0` `--zero` `--empty`	`T` | `t`	load an empty database into the specified register (`T` =twister; `t` =twisted)	
`-i` `--input`	`T` | `t`	load the specified binary database into the specified register (`T` =twister; `t` =twisted). File extension is automatically determined depending on database type (will be `.KPopTwister` or `.KPopTwisted` , respectively, unless file is `/dev/*`)	
`-I` `--Input`	`T` | `t`	load the specified tabular database(s) into the specified register (`T` =twister; `t` =twisted). File extension is automatically determined depending on database type (will be: `.KPopTwister.txt` and `.KPopInertia.txt` ; or: `.KPopInertia.txt` and `.KPopTwisted.txt` , respectively, unless file is `/dev/*`)	
`-a` `--add` `--add-to-twisted`	*binary_file_prefix*	add the contents of the specified binary database to the twisted register. File extension is automatically determined (will be `.KPopTwisted` , unless file is `/dev/*`)	
`-o` `--output`	`T` | `t`	save the database present in the specified register (`T` =twister; `t` =twisted) to the specified binary file. File extension is automatically assigned depending on database type (will be `.KPopTwister` or `.KPopTwisted` , respectively, unless file is `/dev/*`)	
`--precision-for-tables`	*positive_integer*	set how many precision digits should be used when outputting numbers in tabular formats	default= *15*
`-O` `--Output`	`T` | `t`	save the database present in the specified register (`T` =twister; `t` =twisted) to the specified tabular files. File extensions are automatically assigned depending on database type (will be: `.KPopTwister.txt` and `.KPopInertia.txt` ; or: `.KPopInertia.txt` and `.KPopTwisted.txt` , respectively, unless file is `/dev/*`)	

Actions on the database register — Other operations:

Option	Argument(s)	Effect	Note(s)
`-t` `--twist` `--twist-kmers` `--twist-spectra`	*binary_file_prefix*	twist the *k*-mer spectra contained in the specified binary database according to the transformation present in the twister register, and add the results to the database loaded in the twisted register	
`-m` `--metric` `--metric-function`	`flat` | `powers(` *POWERS_PARAMETERS* `)`	where *POWERS_PARAMETERS := INTERNAL_POWER , FRACTIONAL_ACCUMULATIVE_THRESHOLD , EXTERNAL_POWER* *INTERNAL_POWER := non-negative_float* *FRACTIONAL_ACCUMULATIVE_THRESHOLD := fractional_float* *EXTERNAL_POWER := non-negative_float* Set the metric function to be used when computing distances. The `power` transformation is computed as follows: 1. the inertia vector is raised to *INTERNAL_POWER* and normalized; 2. elements are summed in order until *FRACTIONAL_ACCUMULATIVE_THRESHOLD* (a number between 0. and 1.) is reached, while the elements above the threshold are set to zero 3. the resulting vector is raised to *EXTERNAL_POWER* and normalized. Note that `flat` (which is equivalent to `power(0,1,1)` or `power(1,1,0)`) disregards inertia, i.e. it is the same as standard coordinates, while `power(1,1,1)` leaves inertia unchanged, i.e. it is the same as principal coordinates	default= `powers(1,1,1)`
`--distance` `--distance-function`	`euclidean` | `cosine` | `angle` | `minkowski(` *non-negative_float* `)`	set the function to be used when computing distances. The parameter for `minkowski` is the power. Note that: - `euclidean` is the same as `minkowski(2)` ; - `cosine` is the same as (`euclidean` ^2)/2, or 1 - cos(theta); - `angle` is the same as arccos(1 - (`euclidean` ^2)/2), or theta, where theta is the relative angle between the two embeddings	default= `euclidean`
`--distance-normalize` `--distance-normalization`	`true` | `false`	whether to normalize twisted vectors before computing distances. It must be `true` when the distance function is `cosine` or `angle`	default= `false`
`-e` `--embeddings` `--compute-embeddings` `--twisted-to-embeddings`	*tabular_file_prefix*	compute embeddings from the vectors present in the twisted register using the current metric function, distance function and normalization. The result will be written to the specified tabular file. File extension is automatically assigned (will be `.KPopVectors.txt` unless file is `/dev/*`)	
`--distances-summarize-at-most` `--distances-in-summary`	*positive_integer* | `all`	set the maximum number of closest sequences to be printed when summarizing distances. Note that more might be printed anyway in case of ties. The statistics in the summary will be computed on all sequences	default= *2*

Option	Argument(s)	Effect	Note(s)
`-d` `--summarize-distances` `--compute-and-summarize-distances`	*twisted_binary_file_prefix* *summary_file_prefix*	for each vector present in the twisted register, compute distances to all vectors present in the specified twisted binary file (which must have extension `.KPopTwisted` unless file is `/dev/*`) using the current metric function, distance function and normalization; summarize them and write the result to the specified tabular file. File extension is automatically assigned (will be `.KPopSummary.txt` unless file is `/dev/*`)	
`-D` `--summarize-and-output-distances` `--compute-summarize-and-output-distances`	*twisted_binary_file_prefix* *summary_file_prefix*	same as option `-d`, but additionally output the distance matrix in tabular form. File extensions are automatically assigned (will be `.KPopSummary.txt` and `.KPopDMatrix.txt`, unless file is `/dev/*`)	
`--neighbors-index-type` `--neighbors-faiss-index-type`	`flat` | `pq(` *PQ_PARAMETERS* `)` | `hnsw(` *positive_integer* `)`	where *PQ_PARAMETERS := positive_integer , positive_integer* Set the type of Faiss index used to compute nearest neighbors. Parameters for `pq` are: number of subquantizers; bits per subquantizer. Note that the product of the two must be less than or equal to the number of dimensions of the twisted vectors. The parameter for `hnsw` is hyperparameter M. Note that some indices may not be able to return all the existing nearest neighbors	default= `hnsw(32)`
`--neighbors-summarize-at-most` `--neighbors-in-summary`	*positive_integer* | `all`	set the maximum number of closest sequences to be printed when summarizing nearest neighbors. Note that more might be printed anyway in case of ties. The statistics in the summary will be computed on all the neighbors explored according to the policy specified by option `--neighbors-guard-policy`	default= *6*
`--neighbors-guard-policy` `--neighbors-exploration-policy`	`times(` _float_no_less_than_one `)` | `plus(` *non-negative_integer* `)`	set the number of nearest neighbors to be explored when summarizing them. Note that this is greater than or equal to the number of neighbors specified with option `--neighbors-summarize-at-most`. Calling the latter *n*, policy `times(` *m* `)` will explore *m*n* nearest neighbors, while policy `plus(` *m* `)` will explore *m+n* nearest neighbors. The additional neighbors explored are not printed, but used to compute overall statistics	default= `times(2.)`
`-n` `--summarize-neighbors` `--find-and-summarize-neighbors`	*twisted_binary_file_prefix* *summary_file_prefix*	for each vector present in the twisted register, find nearest neighbors among the vectors present in the specified twisted binary file (which must have extension `.KPopTwisted` unless file is `/dev/*`) using the current metric function, distance function and normalization; summarize distances and write the result to the specified tabular file. File extension is automatically assigned (will be `.KPopSummary.txt` unless file is `/dev/*`)	

Actions on the database register — Clustering operations:

`KPopTwistDB` exposes two clustering algorithms over the twisted vectors --- greedy leader (the default) and HDBSCAN* --- both selected by `--clusters-method` and configured by the algorithm-specific `--clusters-greedy-*` / `--clusters-hdbscan-*` knob families. The action itself is `-c` / `--clusters`, which clusters either the k-mers (`T`) or the samples (`t`) in the current twister/twisted register.

Option	Argument(s)	Effect	Note(s)
`--clusters-method`	`greedy` | `hdbscan`	clustering algorithm. `greedy`: greedy-leader clustering (see `--clusters-greedy-*` knobs). `hdbscan`: HDBSCAN* density-based clustering (see `--clusters-hdbscan-*` knobs). Same metric/distance/normalisation pre-scaling as `greedy`.	default= `greedy`
`--clusters-greedy-epsilon`	`firstNN` | `density` | `adaptive`	epsilon-estimation strategy for the greedy-leader algorithm: `firstNN`: Kneedle elbow in sorted FAISS 1-NN distances (*O(n log n)* with HNSW, *O(n²)* with flat); `density`: Kneedle elbow in sorted *dist_star* values, where *dist_star* is the distance maximising *k/V(d_k,D)* for each point (*O(n_sample · n)*, or *O(n²)* when `--clusters-greedy-order density` is also set); `adaptive`: per-point *dist_star* as the absorption threshold (no global epsilon); computes *dist_star* for all *n* points (*O(n²)*) and forces the processing order to ascending *dist_star*, so `--clusters-greedy-order` is ignored in this mode. Handles multi-scale cluster structure that a single global threshold cannot capture. Ignored unless `--clusters-method greedy` is in effect.	default= `firstNN`
`--clusters-greedy-order`	`inertia` | `firstNN` | `density`	order in which points are processed by the greedy-leader clusterer. `inertia`: decreasing row inertia proxy (most informative points first); `firstNN`: increasing 1-NN distance (densest regions first); `density`: increasing *dist_star* (densest regions first, *O(n²)*). Ignored unless `--clusters-method greedy` is in effect, or when `--clusters-greedy-epsilon adaptive` (which forces ascending *dist_star* order).	default= `inertia`
`--clusters-greedy-density-sample-number`	*positive_integer*	number of points randomly sampled for *dist_star* estimation when `--clusters-greedy-epsilon density` and `--clusters-greedy-order` is `inertia` or `firstNN`. When `--clusters-greedy-order density` is also set, all *n* points are used. Ignored unless `--clusters-method greedy` is in effect.	default= *200*

Option	Argument(s)	Effect	Note(s)
`--clusters-greedy-index-type`	`flat` | `hnsw(` *positive_integer* `)`	FAISS index type used for 1-NN estimation and greedy-leader clustering. Ignored unless `--clusters-method greedy` is in effect.	default= `hnsw(32)`
`--clusters-hdbscan-min-cluster-size`	*positive_integer*	minimum cluster size for the `hdbscan` clustering algorithm: smaller groups are absorbed as noise during top-down condensation. Settable independently of `--splits-hdbscan-min-cluster-size`. Ignored unless `--clusters-method hdbscan` is in effect.	default= *5*
`--clusters-hdbscan-min-samples`	*positive_integer*	*k* for the core-distance neighbourhood of the `hdbscan` clustering algorithm. When unset, *k* is taken equal to `--clusters-hdbscan-min-cluster-size`, matching the reference HDBSCAN one-knob ergonomic. Ignored unless `--clusters-method hdbscan` is in effect.	default= *same as* `--clusters-hdbscan-min-cluster-size`
`--clusters-hdbscan-mst-mode`	`auto` | `sparse` | `dense`	minimum-spanning-tree construction strategy for the `hdbscan` clustering algorithm. Same three-mode semantics as `--splits-hdbscan-mst-mode` but settable independently. Ignored unless `--clusters-method hdbscan` is in effect.	default= `auto`
`--clusters-hdbscan-num-neighbors`	*positive_integer*	number of nearest neighbours per point used to build the FAISS k-NN candidate graph for the sparse `hdbscan` MST. When unset, auto-computed as *max(* `min_samples` *+ 1, min(n − 1, 30))*. Ignored unless `--clusters-method hdbscan` and `--clusters-hdbscan-mst-mode sparse` are in effect.	default= *auto*
`--clusters-hdbscan-index-type`	`flat` | `pq(` *PQ_PARAMETERS* `)` | `hnsw(` *positive_integer* `)`	FAISS index type used by the sparse `hdbscan` MST when used as the clustering algorithm. Ignored unless `--clusters-method hdbscan` and `--clusters-hdbscan-mst-mode sparse` are in effect.	default= `hnsw(32)`
`-c` `--clusters`	`T` *kmer_list_file* | `t` *class_file*	apply clustering to the contents of the specified register (`T` =twister, clusters k-mers; `t` =twisted, clusters samples). The algorithm is selected by `--clusters-method` (default `greedy`; see also `hdbscan`). Uses the current metric, distance, and normalization settings. The cluster assignment table is written to stdout. For `T`: the names of representative k-mers are written to *kmer_list_file*, one per line and with no header, ready to be passed to `KPopTwist --keep`. Greedy writes the cluster representatives; HDBSCAN writes one representative k-mer per cluster plus every noise k-mer. For `t`: a two-line tab-separated class file is written to *class_file* (header line of sample names; `CLASS` line of class labels), ready to be passed to `KPopCountDB -m -c CLASS`. Greedy labels each sample as `C@` *representative_name*; HDBSCAN labels assigned samples as `C@` *integer* (cluster id) and outlier samples as `noise`.	

Experimental actions — They may be removed from future versions:

Option	Argument(s)	Effect	Note(s)
`--precision-for-splits`	*positive_integer*	set how many precision digits should be used when outputting splits in plain-text format	default= *10*
`--splits-method`	`gaps` | `centroids` | `hdbscan`	algorithm to use when computing splits from embeddings. `gaps`: per-CA-dimension gap candidates, optionally Kneedle-pre-filtered (see `--splits-gaps-kneedle`). `centroids`: *K*-seed bootstrap of inter-centroid bipartitions (see `--splits-centroids-*`). `hdbscan`: condensed HDBSCAN* clusters emitted as nested splits with persistence-weighted branch lengths (see `--splits-hdbscan-*`).	default= `centroids`
`--splits-at-most` `--splits-keep-at-most`	*positive_integer* | `all`	set the maximum number of phylogenetic splits to be kept when generating them from embeddings	default= *10000*
`-S` `--splits` `--compute-splits` `--twisted-to-splits`	*phylosplits_tabular_file_prefix*	compute phylogenetic splits from the vectors present in the twisted register using the current metric function, distance function and normalization. The result will be written to the specified tabular file. File extension is automatically assigned (will be `.PhyloSplits` unless file is `/dev/*`)	

Miscellaneous options. They are set immediately

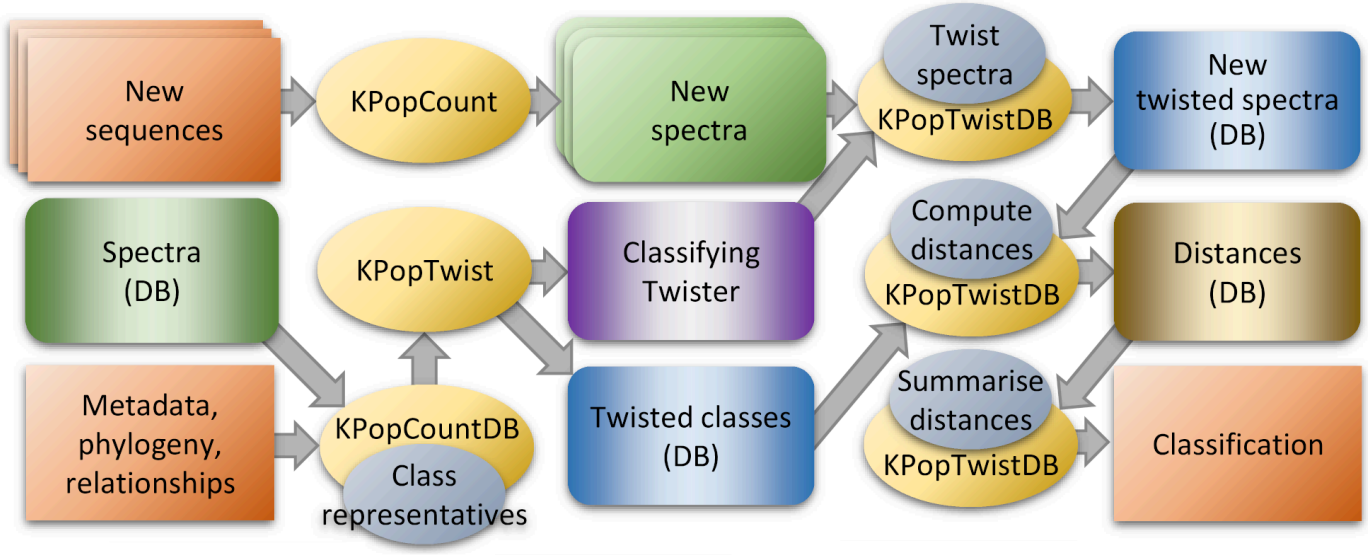
Option	Argument(s)	Effect	Note(s)
`-T` `--threads`	*computing_threads*	number of concurrent computing threads to be spawned (default automatically detected from your configuration)	default= *nproc*
`-v` `--verbose`		set verbose execution	default= *quiet execution*
`-V` `--version`		print version and exit	
`-h` `--help`		print syntax and exit	

5. Examples

By using the programs just described, it is possible to implement a number of interesting high-throughput workflows. We illustrate some examples here - for a more general description, please refer to our Genome Biology paper.

5.1. Sequence classification

One can implement a sequence classifier, starting from either a set of genomic sequences or a deep-sequencing dataset containing multiple samples, by using the following strategy:



Shortly, one first generates a collection of spectra that describe the "classes" understood by the classifier; from the classes a "twister", i.e. a specialised transformation, is derived; the twister is then used to transform new sequences; and distances are computed and summarised between the twisted spectra for the classes and the twisted spectra for the sequences in order to understand what is the class closest to each sequence (possibly with some additional statistical refinement).

Note that the classifier should be generated according to the data type of the input, i.e., you should not use a classifier trained on genomic sequences to process NGS samples or vice-versa. Doing so might occasionally work if contaminations are low and the sequencing bias is reasonably flat along the sequence, but in order to get consistent results significant post-processing might be needed.

5.1.1. Classifier for simulated *M.tuberculosis* sequencing reads

As described in our Genome Biology paper, we simulated sequencing reads for 1,000 *M.tuberculosis* genomes. The data thus generated is large (~127 GB) and hence we are not making it directly available for download. However, the scripts used to (re-)generate it can be found in the `test` directory of this repository.

5.1.1.1. Data preparation

In the following, we assume that the input files derived from the simulation have been organised into directories relative to your current location, and their placement reflects the set (training/test) and sequence cluster each sample belongs to.

So, we'll have two directories,

```
Train/
Test/
```

and each directory will contain subdirectories, one for each sequence cluster:

```
Train/1
Train/2
...
Train/10
```

For instance, directory `Train/8` will contain input files for all the samples used as training data for sequence class `8`, and command

```
ls Train/8
```

will return

```
683_1.fastq  691_1.fastq  699_1.fastq  707_1.fastq  715_1.fastq  723_1.fastq  731_1.fastq  739_1.fastq  747_1.fastq
755_1.fastq  763_1.fastq  771_1.fastq  779_1.fastq  683_2.fastq  691_2.fastq  699_2.fastq  707_2.fastq  715_2.fastq
723_2.fastq  731_2.fastq  739_2.fastq  747_2.fastq  755_2.fastq  763_2.fastq  771_2.fastq  779_2.fastq  685_1.fastq
693_1.fastq  701_1.fastq  709_1.fastq  717_1.fastq  725_1.fastq  733_1.fastq  741_1.fastq  749_1.fastq  757_1.fastq
765_1.fastq  773_1.fastq  781_1.fastq  685_2.fastq  693_2.fastq  701_2.fastq  709_2.fastq  717_2.fastq  725_2.fastq
733_2.fastq  741_2.fastq  749_2.fastq  757_2.fastq  765_2.fastq  773_2.fastq  781_2.fastq  687_1.fastq  695_1.fastq
703_1.fastq  711_1.fastq  719_1.fastq  727_1.fastq  735_1.fastq  743_1.fastq  751_1.fastq  759_1.fastq  767_1.fastq
775_1.fastq  783_1.fastq  687_2.fastq  695_2.fastq  703_2.fastq  711_2.fastq  719_2.fastq  727_2.fastq  735_2.fastq
```

```
743_2.fastq  751_2.fastq  759_2.fastq  767_2.fastq  775_2.fastq  783_2.fastq  689_1.fastq  697_1.fastq  705_1.fastq
713_1.fastq  721_1.fastq  729_1.fastq  737_1.fastq  745_1.fastq  753_1.fastq  761_1.fastq  769_1.fastq  777_1.fastq
689_2.fastq  697_2.fastq  705_2.fastq  713_2.fastq  721_2.fastq  729_2.fastq  737_2.fastq  745_2.fastq  753_2.fastq
761_2.fastq  769_2.fastq  777_2.fastq
```

A similar structure will have been put in place for test data under the `Test/` directory, with files equally split under `Train/` and `Test/` for cross-validation purposes (however, different choices would be possible). This conventional arrangement will be assumed in the rest of this document for all the examples involving `KPop`-based classifiers.

5.1.1.2. Data analysis

In order to analyse sequences in parallel fashion and decrease waiting times, we first prepare a short `bash` script named `process_classes`, as follows:

```
#!/usr/bin/env bash

while read DIR; do
  CLASS=$(echo "$DIR" | awk '{l=split(gensub("[/]+$","",1),s,"/"); print s[l]}')
  echo "Processing class '${CLASS}'..." > /dev/stderr
  ls "$DIR"/*_1.fastq |
    Parallel --lines-per-block 1 -- awk '{l=split($0,s,"/"); system("KPopCount -k 12 -l "gensub("_1.fastq$","",1,s[l])" -p "$0"
        "gensub("_1.fastq$","_2.fastq",1))}' |
    KPopCountDB -k /dev/stdin -R "~." -A "$CLASS" -P -L "$CLASS" -N -P -D --summary -t /dev/stdout 2> /dev/null
done
```

The program `Parallel` can be obtained from the BiOCamLib repository. on which the implementation of `KPop` depends.

The script takes as input a list of directories. For each directory, the script generates the 10-mer spectra for all the files contained in that directory, combines the spectra into an in-memory `KPopCount` database, generates their linear combination, discards the database, and writes the combination to standard output.

So, for instance,

```
echo Train/8 | ./process_classes
```

would process all files present in directory `Train/8`.

Note that the script is implicitly parallelised, in that both `Parallel` and `KPopCountDB` will automatically check for the number of available processors, and start an adequate number of computing threads to take full advantage of them. `KPopCount` is not by itself parallel, but multiple copies of it will be invoked through `Parallel`.

At this point, in order to perform the "training" phase, we need to issue the commands

```
ls -d Train/*/ | Parallel --lines-per-block 1 -- ./process_classes | KPopCountDB -k /dev/stdin -o Classes
KPopTwist -i Classes -o Classes
```

The first command will generate one combined, representative spectrum for each class in the training set, and subsequently, thanks to `KPopCountDB`, combine the spectra for all the representatives into a database having prefix `Classes` and full name `Classes.KPopCounter`.

The second command will "twist" the database, i.e., generate a dataset-specific transformation that turns *k*-mer spectra of the size chosen when generating them (10 in this case) into vectors embedded in a reduced-dimensionality space. The transformation is stored in the file `Classes.KPopTwister`. In addition, `KPopTwist` also produces a file containing the positions of the class representatives in twisted space (`Classes.KPopTwisted`). Such files will be reused in what follows.

Once that has been done, the command

```
ls Test/*/*_1.fastq | Parallel --lines-per-block 1 -- awk '{l=split($0,s,"/"); system("KPopCount -k 12 -l "gensub("_1.fastq$","",1,s[l])" -
        p "$0" "gensub("_1.fastq$","_2.fastq",1))}' | KPopTwistDB -i T Classes -k /dev/stdin -o t Test -v
```

will generate separate *k*-mer spectra for each file in the test set, and twist them according to the "classifying" transformation stored in file `Classes.KPopTwister`. The results will be stored in an additional file, `Test.KPopTwisted`.

All the files generated so far are binary — their content is laid out according to the way OCaml data structures are stored in memory.

Should you be curious to see what the content looks like in human-readable format, you can always use `KPopTwistDB` to convert the file (or any other file generated by `KPopTwist` or `KPopTwistDB`) to a tab-separate textual table as those you can import into, or export from, R. For instance, the command

```
KPopTwistDB -i t Test -O t Test
```

will load a binary file (hence option `-i`) of the "twisted" type (hence option `-i t`) with prefix `Test` (hence option `-i t Test`), i.e., according to the automatic naming conventions enforced by `KPop`, it will load file `Test.KPopTwisted` into the "twisted" register of `KPopTwistDB`. After that, the command will output the content of the "twisted" register in tabular form (hence option `-O t`) to a file with prefix `Test`, i.e., according to `KPop`'s automatic naming conventions, to file `Test.KPopTwisted.txt`. You can see how such naming conventions free you from thinking about files extensions, and avoid name clashes between files having the same prefix but a different content.

The file `Test.KPopTwisted.txt` will contain a header

```
""  "Dim1"  "Dim2"  "Dim3"  "Dim4"  "Dim5"  "Dim6"  "Dim7"  "Dim8"  "Dim9"
```

followed by rows such as

```
"121"   0.461489766036905      0.568113255163704      -0.882699288338637      -0.0272667586657692     0.0581302393092316
-0.0189776114363531     0.00161058296863769     0.0225299502172142      -0.0278201311947722
```

expressing the coordinates of twisted sequences or samples (in this case, sample `121`) in twisted space.

We now have several files in our directory, namely

```
Train/
Test/
Classes.KPopCounter
Classes.KPopTwister
Classes.KPopTwisted
Test.KPopTwisted
Test.KPopTwisted.txt
```

What is left to do in order to classify the sequences in our test set is to compute the distance in twisted space between each sequence (rows of file `Test.KPopTwisted`) and the representative of each equivalence class of the training set (rows of file `Classes.KPopTwisted`); the classification for each given sequence will be the closest equivalence class (provided that the closest and second closest match are separated by some reasonable margin). Computing all pairwise distances in twisted space between sequences and classes also requires transformation values from `Classes.KPopTwister`, and is accomplished by the command

```
KPopTwistDB -i t Classes -i T Classes -d Test -O d Test-vs-Classes -o d Test-vs-Classes
```

which loads the twisted file `Classes.KPopTwisted`, computes pairwise distances with the contents of `Test.KPopTwisted` — the results will be placed in the "distance" register of `KPopTwistDB` —, and writes results into a binary file `Test-vs-Classes.KPopDMatrix` and a tabular file `Test-vs-Classes.KPopDMatrix.txt` (we write to a text here for illustration). File `Test-vs-Classes.KPopDMatrix.txt` will contain a header

```
""      "10"    "1"     "2"     "3"     "4"     "5"     "6"     "7"     "8"     "9"
```

i.e., the list of the training classes, followed by rows of corresponding distances, such as

```
"121"   3.298234166382  3.33901585931896        1.85388698190156        3.35586331250312        3.35677089495605        3.35069768834933
3.31043491848486        3.30792669036762        3.32894004017435        3.31694544810727
```

In this case, for instance, sequence `121` has distance in twisted space of ~1.9 from class `2`, while the distance from all other classes is >3.3. That classifies the sequence as belonging to class `2` (which is correct according to the truth table of the simulation).

In fact, an automated way of summarising distances and finding the n closest ones is implemented in the option `-s` of `KPopTwistDB`, and that is what one would probably use in real life. The command

```
KPopTwistDB -i T Classes -i t Classes -s Test Test-vs-Classes
```

will produce a file `Test-vs-Classes.KPopSummary.txt` made of tab-separated lines, one per sequence, such as

```
"121"   3.18187160005451        0.467075454492746       3.32894004017435        0.0217576481749768      "2"     1.85388698190156
-2.84319076367473       "10"    3.298234166382  0.249130124925674
```

The first 5 fields have a fixed meaning, being:

1. Sequence name
2. Mean of distances from all classes for the sequence being considered
3. Standard deviation of distances from all classes for the sequence being considered
4. Median of distances from all classes for the sequence being considered
5. MAD of distances from all classes for the sequence being considered.

Those are followed by groups of 3 fields (how many groups depends on the parameters given when the summary was created), each field being:

1. Class name
2. Distance from this class for the sequence being considered
3. *z*-score of the distance from this class for the sequence being considered. The groups list the classes closest to the sequence (2 by default). They are sorted by increasing distance.

For instance, in the example line shown above the summary examines sequence `121`, revealing that its mean distance from classes is 3.18; the closest class is `2`, at a distance of 1.85 (corresponding to a *z*-score of -2.84) while the second closest class is `10`, at a distance of 3.30 (corresponding to a *z*-score of 0.25). This line confirms in a statistically more sound way what we had glimpsed by eye above from the complete list of distances.

Alternatively, `Test-vs-Classes.KPopSummary.txt` can be generated using `Test-vs-Classes.KPopDMatrix.txt` with the `KPopTwistDB` option `-i d`, as follows

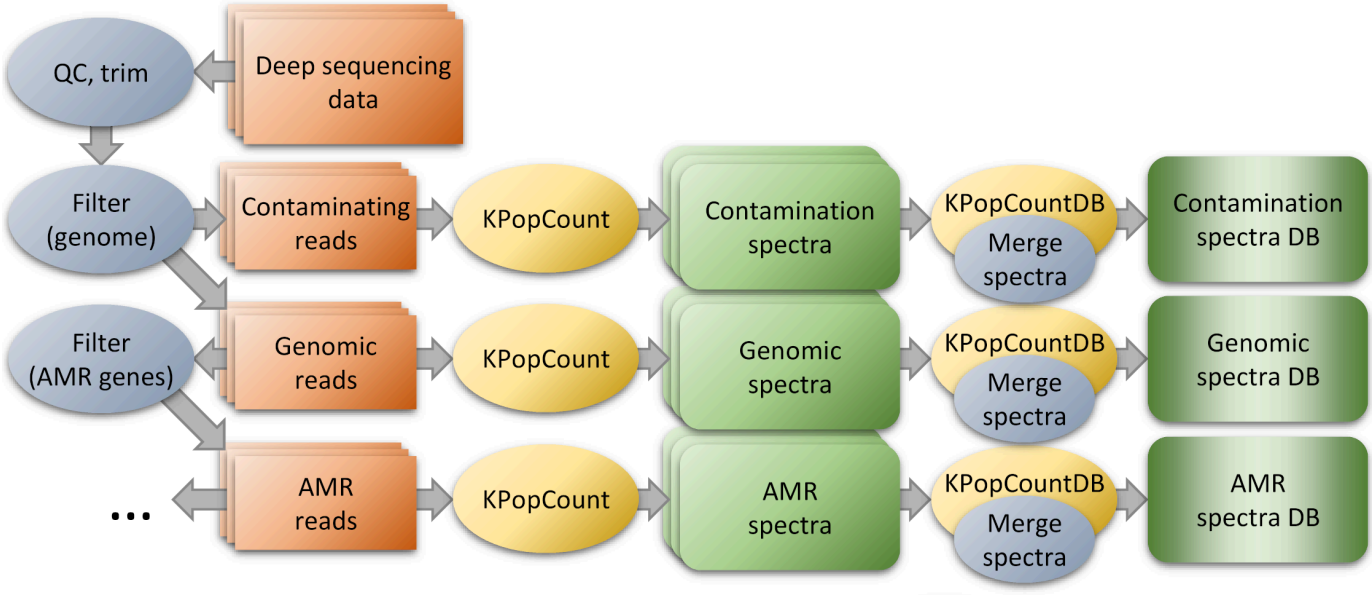
```
KPopTwistDB -i T Classes -i d Test-vs-Classes -S Test-vs-Classes
```

5.1.2. Classifier for deep-sequencing *M.tuberculosis* samples

This example requires a large sequencing dataset available from the Short Read Archive (~1300 samples) to be downloaded and processed. As before, the data is too large to be stored for direct download; however, the lists of SRA identifiers used as a starting point for this experiment can be found in the `test` directory of this repository.

5.1.2.1. Data preparation

As described in our Genome Biology paper, one of the advantages offered by `KPop` is its versatility. In particular, by pre-processing reads one can easily have the method focus on particular regions or features of the genome of interest, as shown in the following figure:



For this example, we are mainly interested in performing a pre-processing step to discard contaminations — i.e., we want to only keep sequencing reads that are likely to originate from *Mycobacterium*. In order to do so, we process each set of sequencing reads through the following script (here we assume to be operating on sample `ERR275184`, which comes as the two paired-end FASTQ files `ERR275184_1.fastq.gz` and `ERR275184_2.fastq.gz`):

```
# First, trim
trim_galore -j 8 --paired ERR275184_1.fastq.gz ERR275184_2.fastq.gz
# Second, filter
FASTools e -p <(zcat ERR275184_1_val_1.fq.gz) <(zcat ERR275184_2_val_2.fq.gz) | FASTools -S | awk -F '\t' 'function round(x){return
        (x%1>=0.5?x+(1-x%1):x-x%1)} BEGIN{MIN_LEN=50; SEGM_LEN=25} {l=length($2); if (l>=MIN_LEN) {n=round(l/SEGM_LEN);
        segm_len=round(l/n); n=round(l/segm_len); split($1,s," "); for (i=1;i<=n;++i) {lo=1+segm_len*(i-1); hi=(i==n?l:segm_len*i); print
        "@"(i==1?s[1]"~"$2"~"$3:s[1])"\n"substr($2,lo,hi-lo+1)"\n+\n"substr($3,lo,hi-lo+1)}}}' | gem3-mapper -I
        Mycobacterium.CompleteGenomes.gem -t ${__CPUS__} -F MAP | awk -F '\t' 'BEGIN{current=""} function process_current(){if
        (current!="") print hits"\t"current} {l=split($1,s,"~"); if (l==3) {process_current(); current=gensub("~","\t","g",$1); hits=0} if
        ($4~"^(|0:|0:0:)[1-9]") ++hits} END{process_current()}' | awk -F '\t' -v FIRST=ERR275184_1.fastq -v SECOND=ERR275184_2.fastq
        'BEGIN{MIN_HITS=3; current=""; found=0} function process_current(){if (current!=""&&hits>=MIN_HITS&&found==2) {print first >
        FIRST; print second > SECOND}} {if ($2!=current) {process_current(); current=$2; found=1; hits=$1; first="@"$2"\n"$3"\n+\n"$4}
        else {++found; hits+=$1; second="@"$2"\n"$3"\n+\n"$4}} END{process_current()}'
# Third, merge
flash -m 20 -M 500 ERR275184_1.fastq ERR275184_2.fastq -o ERR275184
# Fourth, count k-mers
KPopCount -l ERR275184 -s ERR275184.extendedFrags.fastq -p ERR275184.notCombined_1.fastq ERR275184.notCombined_2.fastq > ERR275184.k12.txt
```

The script requires several additional components, namely:

- Trim Galore and dependencies. It is run at the very beginning of the script to do quality and adapter trimming on the reads
- `FASTools`. It is a Swiss-knife tool to manipulate FASTA/FASTQ files. It can be obtained from the BiOCamLib repository. on which the implementation of `KPop` depends
- The GEM mapper. It is a high-performance and very accurate aligner with an expressive native output format that we exploit in our script to quickly select matches
- FLASH. It is a fast program to merge partially overlapping FASTQ paired-end reads. While our approach is not very sensitive to their presence, merging them helps keeping the overall *k*-mer normalisation more even, and it is hence considered good practice.

The main idea behind the script is to split each read into small segments (`SEGM_LEN`, i.e., 25 nt in the code above), map each chunk into a *Mycobacterium* "pangenome", and keep the pair only if a sufficient number of segments (`MIN_HITS`, i.e, 3 in the code above) align to the pangenome. The sequences we used for the pangenome are listed in the `test` directory of this repository and can be downloaded from the SRA — we'll assume that they have all been concatenated into file `Mycobacterium.CompleteGenomes.fasta`. Once that has been done, an index for the pangenome can be generated with the command

```
gem3-indexer -i Mycobacterium.CompleteGenomes.fasta -o Mycobacterium.CompleteGenomes
```

That will generate the file `Mycobacterium.CompleteGenomes.gem` appearing in the script.

After each read has been split into chunks and aligned to the pangenome, the input of the first `awk` command executed after `gem3-mapper` will have FASTQ record in tabular format, with the number of mapping chunks coming before it:

```
4       ERR275184.16    NGCACTCGGGGGCATGCCGATGCCCAAGGCAGTGGTCTGGGCGCTGCATGAGCACATCTTGGGCGCCAATCCGGCGGTATGGATGTACGCCGGCGGCGCGG
#1=DFFFFHHHHHJJJJJJJJJIJJICHIJJJHJHHHHHFFEDDDDDDDCCCDDDDDDDDCC@BD@BDDDDDDDDDDBBBDDDDDED?BDDDBDDDD<99
2       ERR275184.16    ACCCGAGGTAATGAACCGCTTGACGCCGTCGATGTGCCAGGACCCGTCGGCCTGTTGG      B?<DA:D60+:
<<DGIIBEIIIACGHDE:FG0BF=@FCHCHGDG;B<;A/3;';>@:?
4       ERR275184.17    TTGAACGCTACCCATCTCTTTATGGCGTGCCGTCGTGACCGACCCAAGGAGCAGTACACGATCGATTTTTCGACGGTTGCCGCGCTCGATTACGTACCGCT
CCCFFFFFHHHGGJJJJJJJJJJEHIJIJJJJJJJJJJJJIJIIIGGFFFDFDDEEEEDCBDDBDDDDDDDA>?@BBBBB<A>BBDDB@BCCBA<?@BBB@9
4       ERR275184.18    NACACCCCGACAATCTGCGGGCGCAGGTTGGGGTCGGGGAAGCCGCTGCGCAGCGGCGCGCCCGACGGGTCCTTGAGCCCGACGAAGTTGGCGAAGGTGCC
#1BDDFFFHHHHHJJJJJJJJJJJIJJJIIIHHFFDCDDDD@@?BDDDDDDDDDDDDDDDDDD;7@BB5;5>BCD:>CCDDDDDB5<?83:>@B<B<<?CD9
1       ERR275184.18    CAATTTGCAGGCGTTGGTCGTCGTCGTCAGCGCCCAGCGCGTGGACGTCACCG   @@@DDFDDFFFDFG6)+29B8?18??)0?@*877<6(8;;8993;639><8C@
3       ERR275184.19    GTCGCAGCCGGGACGCTGCTATGGCTCGGACGTGTCGAAATTCGGGTCACCGCCGGCTCAGCGGATGGAGCC        CCCFFFFFHHHHHJJJJJJJJJGIJJJEHBEE@C?
A86@D;C;=BD58?ACB?B5@B<;39A7<-55?C?9?
```

and the second `awk` commands counts the sum of aligned segments and decides whether or not to keep the pair.

However, equivalent techniques that discard extraneous reads using different approaches or programs would also be acceptable.

5.1.2.2. Data analysis

Once reads have been pre-processed and their *k*-mer spectrum generated for each sample, data analysis proceeds along the lines of the simulated *M.tuberculosis* example above — we assume that the spectra have been placed in a `Train` and `Test` directory and separated into subdirectory according to their class, as per the convention previously described. We then generate representative spectra for the classes; twist them; and use the resulting transformation to twist the test sequences, processing each directory in parallel. The results are then collected in the database `Test.KPopTwisted`.

```
ls -d Train/*/ | awk '{print substr(gensub("Train/","",1),1,length($0)-7)}' | Parallel -l 1 -t 4 -- awk '{CLASS=$0; system("cat
        Train/"CLASS"/*.txt | KPopCountDB -k /dev/stdin -R \"~.\" -A "CLASS" -L "CLASS" -N -D -v -t "CLASS")}'
cat M_*.KPopCounter.txt | KPopCountDB -k /dev/stdin -o Classes -v
KPopTwist -i Classes -o Classes -v
ls -d Test/M_*/ | Parallel -l 1 -t 4 -- awk '{system("cat "$0"*.k12.txt | awk -F \047\\t\047 \047{if ($1==\"\") print
        $0\"\\001"substr($0,6,length($0)-6)"\"; else print}\047 | KPopTwistDB -i T Classes -k /dev/stdin -o t "substr($0,1,length($0)-1)"
        -v")}'
KPopTwistDB $(ls Test/*.KPopTwisted | awk '{split($0,s,"[.]"); printf " -a t "s[1]}') -o t Test
```

Note that in this example we annotate the name of each test sequence with its class, by adding to it a non-printable character `\001` followed by the class (`KPop` is fine with non-printable characters in labels, provided that they are not `\000`). We'll do the same below for the training sequences below, so as to be able to count the majority class when we use a *k*-NN approach.

As detailed in our Genome Biology paper, for this example we explored three different classifiers. Although they all provide very good performance, these alternative approaches are easy to implement using `KPop` tools and illustrate the versatility of our framework.

Two-class workflow

This approach works exactly as in the simulated *M.tuberculosis* example above:

```
KPopTwistDB -i T Classes -i t Test -d Classes -o d Test-vs-Classes -s Test Test-vs-Classes -v
cat Test-vs-Classes.KPopSummary.txt | awk '{print gensub("\001","\"\t\"","g")}' > RESULTS-2C.txt
```

Note the use of the distance capability provided by `KPopTwistDB`, as in the previous example, which produces results with the format explained above. We also post-process such results to split the name of the test sequence and its annotated class into two separate columns — as mentioned before, we had joined them through a non-printable character `\001` that we now turn into `\t`.

k-NN approach

In this case, we implement the classifier by using a *k*-NN approach, i.e. by performing a majority call among the *k* nearest neighbours found for each test sequence - the result is the class which is most represented in the *k* closest data points belonging to the training set. In order to be able to do so, we need to separately compute the twisted spectrum of each training sequence — we also collect them into a database named `Train`:

```
ls -d Train/M_*/ | Parallel -l 1 -t 4 -- awk '{system("cat "$0"*.k12.txt | awk -F \047\\t\047 \047{if ($1==\"\") print
        $0\"\\001"substr($0,7,length($0)-7)"\"; else print}\047 | KPopTwistDB -i T Classes -k /dev/stdin -o t "substr($0,1,length($0)-1)"
        -v")}'
KPopTwistDB $(ls Train/*.KPopTwisted | awk '{split($0,s,"[.]"); printf " -a t "s[1]}') -o t Train
```

We then use the distance capability of `KPopTwist` as before. However, in this case we alter the default to output information about 5 nearest neighbours rather than 2:

```
KPopTwistDB $(ls Test/*.KPopTwisted | awk '{split($0,s,"[.]"); printf " -a t "s[1]}') -o t Test -d Train -o d Test-vs-Train --keep-at-most
        5 -s Test Test-vs-Train -v
```

This produces the usual digest of the distance matrix containing, in this case, the 5 training sequences that are closest to the test sequence (we adopt a 5-NN classification approach in this example, but of course the number of neighbours could be chosen to be different). At this point we are almost done, but we still have to parse the digest in order to compute how many of the nearest neighbours belong to the most frequent class, which translates into a slightly more complex script:

```
cat Test-vs-Train.KPopSummary.txt | awk '{print gensub("\001","\"\t\"","g")}' | awk '{delete c; delete o; n=0; for (i=8;i<=NF;i+=4) {++n;
        if (!($i in c)) o[length(c)+1]=$i; ++c[$i]} max=0; max_class=0; l=length(o); for (i=1;i<=l;++i) {if (c[o[i]]>max) {max=c[o[i]];
        max_class=o[i]}} print $0"\t"max_class"\t"(max/n)}' > RESULTS-kNN.txt
```

Notably, we solve ties by selecting the class that appears first, i.e. which has a sequence that is closer to the test sequence.

Random forest approach

This is a more sophisticated approach that uses random forests to implement the predictor. Note that this is possible because `KPop` is capable of producing an explicit embedding of the sequences as a vector in twisted space. As we are going to use an external R library to implement the predictor, we first have to export to a tabular text file the coordinates of the sequences in twisted space — which we can do with `KPopTwist` — and then, once more, de-mangle the class information into a separate column:

```
KPopTwistDB -i t Train -O t Train -i t Test -O t Test
cat Train.KPopTwisted.txt | awk '{print gensub("\001","\"\t\"","g")}' | awk -F '\t' 'BEGIN{OFS="\t"} {if (NR==1) {$1="\"Lineage\""};
        print}' > Train.Truth.KPopTwisted.txt
cat Test.KPopTwisted.txt | awk '{print gensub("\001","\"\t\"","g")}' | awk -F '\t' 'BEGIN{OFS="\t"} {if (NR==1) {$1="\"Lineage\""} else
        $2="\"\""; print}' > Test.Truth.KPopTwisted.txt
```

The files `Train.Truth.KPopTwisted.txt` and `Test.Truth.KPopTwisted.txt` will now contain the coordinates of train and test sequences in twisted space, respectively. We then run the following `R` commands:

```
library(randomForest)
train<-read.table("Train.Truth.KPopTwisted.txt")
test<-read.table("Test.Truth.KPopTwisted.txt")
rf<-randomForest(Lineage~.,data=train,mtry=4)
write.table(as.data.frame(predict(rf,test)),"RF.txt",sep="\t")
```

which will produce a file `RF.txt` with the preliminary results of the predictor.

And finally, this command re-annotates the sequences with their original class for comparison purposes:

```
cat RF.txt | awk -F '\t' 'BEGIN{while (getline < "NGS-TB-Test.txt") t["\""$1"\""]="\""$2"\""} {if (NR>1) print $1"\t"t[$1]"\t"$2}' >
        RESULTS-RF.txt
```

generating a final result `RESULTS-RF.txt`. The file `NGS-TB-Test.txt` contains the original annotation for each test sequence, and is available from the `test` directory of this repository.

It should be noted that the structure of `RESULTS-RF.txt` is simpler than that of the final files produced by the previous methods. It contains lines such as

```
...
"ERR046839"     "M_tuberculosis_L4"     "M_tuberculosis_L4"
"ERR046933"     "M_tuberculosis_L5"     "M_tuberculosis_L5"
"ERR072050"     "M_tuberculosis_L4"     "M_tuberculosis_L4"
"ERR1023314"    "M_tuberculosis_L4"     "M_tuberculosis_L4"
"ERR1023338"    "M_tuberculosis_L3"     "M_tuberculosis_L3"
"ERR1023449"    "M_tuberculosis_L2"     "M_tuberculosis_L2"
"ERR1034887"    "M_tuberculosis_L4"     "M_tuberculosis_L4"
...
```

each one reporting samples name, original class, and classification generated by random forest — there is no additional statistical information reported in the default output generated by `randomForest`.

5.1.3. Classifier for SARS-CoV-2 sequences (Hyena)

This is a rather more complex example, that showcases many of the good qualities of `KPop` (mainly its being high-throughput and accurate). It's also not for the faint of heart, in that it requires large amounts of disk space and computing time — at the time of this writing, the file containing all COVID-19 sequences made available on GISAID has an uncompressed size of ~303 GB, and counting. (And we know that technically we should call them SARS-CoV-2 sequences, but in this section we'll sloppily say COVID-19 instead.) In fact, most of the needed preprocessing does not really have much to do with `KPop`, but we work out the complete example anyway for clarity's sake. Feel free to skip it if you are only interested in the `KPop` part.

5.1.3.1. Data preparation

▶ Preparing input data for the SARS-CoV-2 classifier

5.1.3.2. Data analysis

Once the preparation stage has been completed, the analysis proceeds exactly as in the *M.tuberculosis* example above.

First, we compute the spectra for the class representatives using `KPopCount` and `KPopCountDB`:

```
ls Train/*.fasta | Parallel -l 1 -- awk '{l=split($0,s,"/"); class=substr(s[l],1,length(s[l])-6); system("KPopCount -k 10 -l "class" -f
        Train/"class".fasta")}' | KPopCountDB -k /dev/stdin -o Classes
```

This produces a `Classes.KPopCounter` file which is ~2.0 GB. The command

```
KPopCountDB -i Classes --summary
```

confirms that indeed this database contains the spectra for all classes:

```
[Vector labels (1636)]: 'A.11' 'A.12' 'A.15' 'A.16' 'A.17' 'A.18' 'A.19' 'A.1' 'A.21' 'A.2.2' 'A.22' 'A.23.1' 'A.2.3' 'A.23' 'A.2.4' 'A.24'
'A.2.5.1' 'A.2.5.2' 'A.2.5.3' 'A.2.5' 'A.25' 'A.26' 'A.27' 'A.28' 'A.29' 'A.2' 'A.30' 'A.3' 'A.4' 'A.5' 'A.6' 'A.7' 'A.9' 'AA.1' 'AA.2'
'AA.3' 'AA.4' 'AA.5' 'AA.6' 'AA.7' 'AA.8' 'AB.1' 'AC.1' 'AD.1' 'AD.2.1' 'AD.2' 'AE.1' 'AE.2' 'AE.3' 'AE.4' 'AE.5' 'AE.6' 'AE.7' 'AE.8'
'AF.1' 'A' 'AG.1' 'AH.1' 'AH.2' 'AH.3' 'AJ.1' 'AK.1' 'AK.2' 'AL.1' 'AM.1' 'AM.2' 'AM.3' 'AM.4' 'AN.1' 'AP.1' 'AQ.1' 'AQ.2' 'AS.1' 'AS.2'
'AT.1' 'AU.1' 'AU.2' 'AU.3' 'AV.1' 'AW.1' 'AY.100' 'AY.101' 'AY.102.1' 'AY.102.2' 'AY.102' 'AY.103.1' 'AY.103.2' 'AY.103' 'AY.104' 'AY.105'
'AY.106' 'AY.107' 'AY.108' 'AY.109' 'AY.10' 'AY.110' 'AY.111' 'AY.112.1' 'AY.112' 'AY.113' 'AY.114' 'AY.116.1' 'AY.116' 'AY.117' 'AY.118'
'AY.119.1' 'AY.119.2' 'AY.119' 'AY.11' 'AY.120.1' 'AY.120.2.1' 'AY.120.2' 'AY.120' 'AY.121.1' 'AY.121' 'AY.122.1' ... 'P.6' 'P.7' 'Q.1'
'Q.2' 'Q.3' 'Q.4' 'Q.5' 'Q.6' 'Q.7' 'Q.8' 'R.1' 'R.2' 'S.1' 'U.1' 'U.2' 'U.3' 'V.1' 'V.2' 'W.1' 'W.2' 'W.3' 'W.4' 'XA' 'XB' 'XC' 'Y.1'
'Z.1'
[Meta-data fields (0)]:
```

According to the usual procedure, we then twist the class representatives with `KPopTwist`:

```
KPopTwist -i Classes -o Classes -v
```

and we use the resulting classifying twister to twist and accumulate the test sequences into file `Test.KPopTwisted` with `KPopTwistDB`:

```
cat Test/*.fasta | awk 'BEGIN{ok=1} {if ($0~"^>") {ok=!($0 in t); t[$0]} if (ok) print}' | Parallel -l 2 -- awk '{if (NR==1)
        {job="KPopCount -k 10 -l \""substr($0,2)"\" -f /dev/stdin"; print $0 |& job} else {print $0 |& job; close(job,"to"); while (job |&
        getline) print $0}}' | KPopTwistDB -i T Classes -k /dev/stdin -o t Test -v
```

Note that right at the beginning of the script, with the part

```
awk 'BEGIN{ok=1} {if ($0~"^>") {ok=!($0 in t); t[$0]} if (ok) print}'
```

we are removing repeated sequences (yes, apparently there are repeated sequences in GISAID 😲) as `KPopTwistDB` is unhappy with them and will bail out if it finds some in the input.

Also, as there are ~650K sequences to be classified, doing so will take long (~14h on the node I'm using for these tests). So, you might wish to further parallelise the process, as I did, by splitting the input into smaller files and processing them separately on different nodes. After that, you can merge together all the pieces with a command such as

```
KPopTwistDB -a t Test.aa -a t Test.ab ... -o t Test -v
```

The final size of the file `Test.KPopTwisted` containing all the ~650K twisted COVID-19 sequences in the test set is ~8.4 GB.

At this point, the analysis proceeds exactly as in the case of the previous section, with a command such as

```
KPopTwistDB -i T Classes -i t Test -d Classes -o d Test-vs-Classes -v
```

that allows us to compute all the distances of each test sequence from each of the "training" classes. However, given that in this case there is a very large number of distances to be computed, another and perhaps more appropriate strategy would have been not to merge the twisted test files into a single `Test.KPopTwisted` file, but rather to compute the distances from the twisted classes for each of the chunks with commands such as

```
KPopTwistDB -i T Classes -i t Test.aa -d Classes -o d Test-vs-Classes.aa
```

and then merge together all the distance files, as in

```
KPopTwistDB -a d Test-vs-Classes.aa -a d Test-vs-Classes.ab ... -o d Test-vs-Classes -v
```

One way or another, once a file `Test-vs-Classes.KPopDMatrix` containing all the distances has been generated, we can run

```
KPopTwistDB -i d Test-vs-Classes -s Test.aa Test-vs-Classes -v
```

in order to produce the usual textual summary of the distances. The resulting file will be ~118 MB in size.

And quite likely, one would also wish to run something like the following commands:

```
cat Test-vs-Classes.KPopSummary.txt | awk 'function remove_spaces(name,     s){split(name,s,"/"); return gensub("[
        _]","","g",s[1])"/"s[2]"/"s[3]} BEGIN{nr=0; while (getline < "lineages.csv") {++nr; if (nr>1) {split($0,s,",");
        t[remove_spaces(gensub("^BHR/","Bahrain/",1,s[1]))]=s[2]}}} {printf $1"\t\""t[substr($1,2,length($1)-2)]"\""; for (i=2;i<=NF;++i)
        printf "\t"$i; printf "\n"}' > Test-vs-Classes.KPopSummary.Truth.txt
cat Test-vs-Classes.KPopSummary.Truth.txt | awk -F '\t' 'function strip_quotes(s){return substr(s,2,length(s)-2)} {one=strip_quotes($2);
        two=strip_quotes($7); print one"\t"two"\t"(substr(two,1,length(one)+1)) > "/dev/null"; if
        ($2!=$7&&substr(two,1,length(one)+1)!=one".") ++ko; else ++ok}
        END{printf("%d\t%d\t%.3g%%\t%d\t%.3g%%\n",ok+ko,ok,ok/(ok+ko)*100,ko,ko/(ok+ko)*100)}'
```

to, first, annotate the summary with the original "true" classification of the sequences, and, second, generate a one-line summary of the results. The last command will produce something like:

```
641847    611770    95.3%    30077    4.69%
```

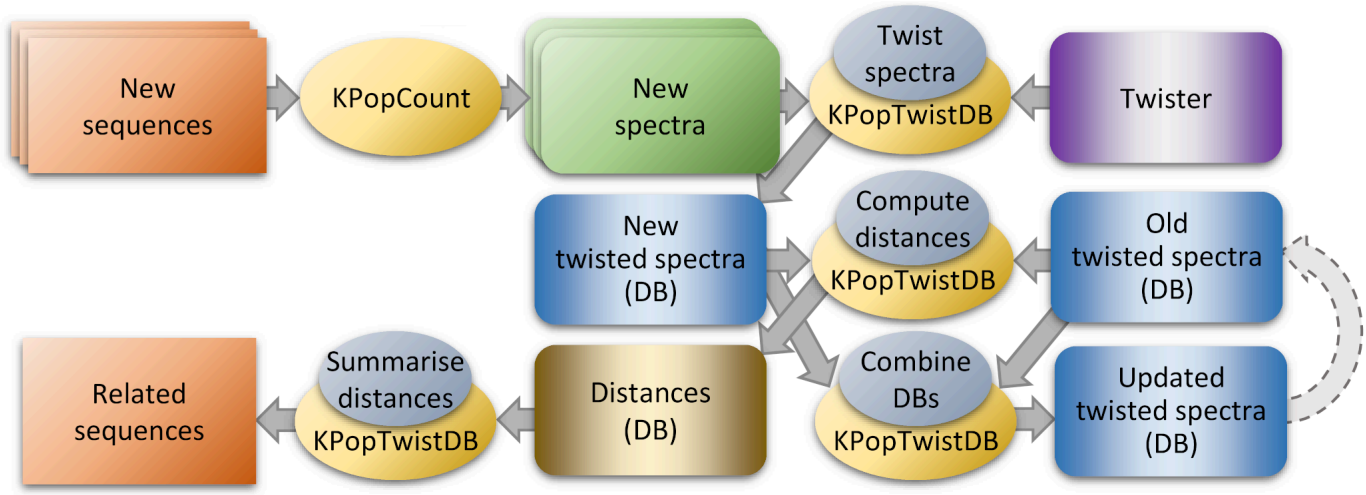
meaning that, of the 641,847 sequences present in the `Test` set, 611,770 (95.3%) were correctly classified, while 30,077 (4.69%) were not.

Note that, as discussed in more detail in our Genome Biology paper, in this example many classes are actually heterogeneous collections of more than one subtree. One might then wish to consider more sophisticated classification methods, such as random forests or others.

5.2. Relatedness engine

The previous examples have all showcased `KPop`'s capability to encode sequences as vectors in twisted space and find the closest neighbours there. This correctly suggests that by using `KPop` one can also easily generate a "relatedness engine" out of a classification — i.e., a system finding the most similar sequences in a large database.

The overall strategy is illustrated in the following figure, which assumes that we have already generated a classifying twister (purple box) and a database of twisted sequences (second box from the top on the right) to start with:



Briefly, we can generate and twist new spectra (top left), find out their nearest neighbours in the database of twisted sequences (central line and bottom left), and add the new twisted spectra to the database (bottom right).

Here we illustrate the approach taking the previous exercise on COVID-19 as a starting point. For the sake of simplicity, we'll take as existing database the twisted test sequences (half of the GISAID database) in the file `Test.KPopTwisted`. The classifier will be the one we generated in the previous section and contained in the file `Classes.KPopTwister`. The command

```
cat Train/C.36.3.fasta | fasta-tabular | shuf | head -1 | awk -F '\t' '{print $1 > "/dev/stderr"; print ">"$1"\n"$2}' | KPopCount -k 10 -f
        /dev/stdin -l "C.36.3" | KPopTwistDB -i T Classes -i t Classes -k /dev/stdin -d Test --keep-at-most 300 -s Test Related
```

will then randomly select one sequence from the `C.36.3` lineage and find the 300 sequences closest to it. A summary in the usual format will be output to file `Related.KPopSummary.txt`. Note that loading this specific database in memory is going to take long as the file is ~8.4 G, but one only needs to do it once when sequences are processed in batches.

5.3. Pseudo-phylogenetic trees

> 🚧 Coming soon! 🚧